Conversación con Claude — TPI MS2: cierre de diagnóstico y tests, y refactoring (Fase 3)

Integrante: Cristian Morales
Herramienta: Claude (Anthropic), sesión de trabajo en claude.ai
Período: 31/10/2026 20:44 a 07/11/2026 23:10 (hora de Argentina) — 96 mensajes
Fuente: exportación oficial de datos de la cuenta (Settings → Privacy → Export data), conversación «Conftest construction»
Nota: se transcribe el texto visible de cada mensaje. Se omiten el razonamiento interno del modelo y el detalle técnico de las herramientas; en su lugar se indica la acción realizada (archivos enviados, comandos ejecutados, accesos al repositorio). El acceso al repositorio fue de solo lectura: la IA no hizo commits ni push. En esta parte la IA trabajó sobre una copia local de solo lectura: armó y verificó los cambios ahí, y los commits en el repositorio los hizo el usuario, aplicando los archivos o el paso a paso. Se omiten los intercambios sobre la exportación de las conversaciones, y los intercambios y fragmentos de mensajes que no corresponden a estas etapas. Las capturas adjuntas no se reproducen.
Parte: 4 — Revisión de los diagnósticos y tests del grupo, tags `etapa-diagnostico` y `etapa-tests`, estructura del proyecto y cambios transversales, refactoring de estudiantes y responsables y revisión de los refactors del grupo. Viene de «cristian_claude_03_pruebas.pdf».

Usuario (Cristian Morales) 31/10/2026 20:44

revisa como quedo la branch

Claude 31/10/2026 20:45

▸ *Ejecutó un comando: Revisar la rama actualizada*

Revisé la rama: está todo en orden.

- **Los commits:** los dos nuevos (`c179ec3` con el uso de IA y `de21cd5` con la transcripción) están subidos, y el PR #4 apunta al último.
- **Los tests:** bajé la rama desde GitHub y corrí los tests: `13 passed`.
- **La declaración de uso de IA** dice "casos 1 a 6, 10, 11 y 12".
- **El PDF** que subiste es idéntico al que te mandé: la versión de 41 páginas, sin la parte de la exportación.
- **No se coló ningún** `__pycache__`, y la rama está al día con `dev`, así que no hay conflictos.

El PR suma 8 archivos: `pytest.ini`, `conftest.py`, los dos archivos de tests, el documento de la Fase 2, la declaración de uso de IA, la captura y la transcripción.

Tu parte de la Fase 2 está completa. Solo falta poner el tag `etapa-diagnostico`, y en el repo todavía no está: solo existe `estado-inicial`. Para ponerlo hace falta que:

- se mergee el diagnóstico de Javier (PR #7);
- suban y se mergeen los de Gaston y Bruno, que todavía no tienen rama;
- le avises a Bruno que el `conftest.py` está en `test/personas`, porque sus tests van a depender de él.

Cuando esté el tag, pasás el PR #4 de draft a listo.

Usuario (Cristian Morales) 31/10/2026 20:57

https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta/pull/7/changes

revisa el pr de Javi, le falta algo? esta ok?

Claude 31/10/2026 20:58

▸ *Ejecutó un comando: Revisar los commits y cambios del PR #7 de Javier*

▸ *Ejecutó un comando: Leer los documentos de Javier y compararlos con el formato del grupo*

▸ *Ejecutó un comando: Verificar la base, los conflictos y las evidencias del PR #7*

▸ *Ejecutó un comando: Revisar el PDF de uso de IA de Javier*

Lo revisé. **Está bien y se puede aprobar.** No veo nada que bloquee el merge: lo que encontré son ajustes de coherencia en la tabla de resultados.

Lo que está bien

- **Está completo:** ficha de smells, resultados de la Fase 1, 20 capturas, declaración de uso de IA y la transcripción en PDF (11 páginas).
- **Las evidencias coinciden:** cada captura citada en la tabla existe, y no sobra ninguna.
- **El diagnóstico es sólido.** El hallazgo 3 es muy bueno: muestra que `PEND → ASIG` por `/estado` nunca puede tener éxito, porque `asignar` ya cambia el estado. El 6 (las reasignaciones no quedan en el historial) también es un defecto real y está comprobado.
- **Es coherente con lo tuyo.** Propone unificar su hallazgo 1 con tu hallazgo 3 y su 5 con tu 5, y marca qué tiene que coordinar con Gaston y con Bruno.
- **No da problemas en el repo:** está basado en el `dev` actual y no tiene conflictos.

Lo que convendría ajustar (no bloquea)

1. **El caso 10 se contradice.** En la tabla figura como **"Sí"**, pero la misma fila dice que no se agregó la línea en `notificaciones.log`. Además, las conclusiones lo cuentan como defecto ("asignar no notifica al estudiante") y el hallazgo 3 lo usa como evidencia. Tendría que ser **"Parcial"**: asigna bien, pero no notifica.
2. **La justificación del caso 19 es incompleta.** Dice "código correcto, pero el mensaje es poco claro". El problema de fondo es el **formato**: responde con `detail` y el resto de los errores con `error`. Por eso está como evidencia del hallazgo 1. Es el mismo caso que tu 12; conviene que los dos lo expliquen igual.
3. **La redacción del caso 12c no es precisa.** Dice "el código está bien, pero no el mensaje", pero lo que falla es el **historial**, que no registra la reasignación. El mensaje está bien.
4. **El caso 1 aparece como evidencia del hallazgo 1**, pero está evaluado como "Sí". Tiene sentido como contraste con el caso 2 (404 en un endpoint y 200 en el otro), pero conviene aclararlo en la ficha para que no parezca un error.

Cómo responder el PR

Te recomiendo **aprobar con comentarios**: los cuatro puntos son menores y Javier puede corregirlos en un commit más antes del merge, o después, en otra rama. Si preferís que lo corrija antes, poné "Request changes" solo por el punto 1, que es el único donde el documento se contradice a sí mismo.

¿Querés que te arme el texto del comentario para pegarlo en el PR?

Usuario (Cristian Morales) 02/11/2026 10:26

https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta/pull/10/changes como ves este pr

Claude 02/11/2026 10:27

▸ *Ejecutó un comando: Revisar los commits y cambios del PR #10 de Gaston*

▸ *Ejecutó un comando: Leer los documentos de diagnóstico de Gaston*

▸ *Ejecutó un comando: Verificar las evidencias de Gaston*

Lo revisé. **El diagnóstico está muy bien. Lo que falla es la declaración de uso de IA.** Ahí yo pediría cambios antes de aprobar.

Lo que está bien

- **El contenido es sólido.** Tiene 17 casos con evidencia, y cada captura citada existe. Encontró dos 500 nuevos: con `descripcion` nula o numérica, el `len()` de la línea 144 falla. También detectó `True in [1, 2, 3]`, que es muy bueno para la defensa, y la inconsistencia entre la prioridad inválida (que se rechaza) y el canal inválido (que se corrige en silencio).

- **La ficha está muy completa.** Agregó las columnas "Transversal" y "Refactoring propuesto", y las referencias cruzadas son correctas: su 4 corresponde a tu 3 y al 1 de Javier, su 5 a los 5 de los dos, su 8 a tu 8 y al 9 de Javier. Deja asegurado el **Introduce Parameter Object** (hallazgo 1), que es el refactoring que le tocaba.
- **No da problemas en el repo:** no está basado en el `dev` más reciente, pero no tiene conflictos, porque son archivos nuevos.

Lo que hay que corregir

1. **`docs/uso-ia/gaston.md` está roto.** A la segunda fila le falta la última columna (Evidencia) y el `|` final, y el `## Detalle` quedó pegado a la tabla. En GitHub se va a ver mal.
2. **El diagnóstico no tiene evidencia de uso de IA.** La fila dice grado **Alto** (la IA propuso los casos y armó el borrador de la ficha y de la tabla), pero no cita ninguna evidencia. Además, `gaston_registro_ia.md` solo registra dos pedidos (la plantilla y el panorama), ninguno del diagnóstico. Javier, Bruno y vos tienen la transcripción en PDF. Que la exporte igual (Settings → Privacy → Export data). Si no se puede, que lo registre en el `.md` y lo diga explícitamente.
3. **No queda claro qué redactó él.** Javier y vos aclaran que el impacto, el riesgo, la prioridad y el principio los escribieron ustedes. En la defensa le van a preguntar. Que lo deje escrito, sea cual sea la respuesta.
4. **La nota en `gaston_resultados_fase1.md` corta la tabla.** La nota sobre las solicitudes 7 y 11 está metida en medio de "Datos iniciales relevantes", y la fila "Tipos válidos" queda afuera. Hay que moverla debajo de la tabla.

Temas para acordar en el grupo (no son errores de Gaston)

- **Qué código corresponde a un recurso del body que no existe.** Gaston usa **422** para el estudiante inexistente y lo justifica. Javier usa **404** para el responsable inexistente (su caso 7). Hay que elegir uno antes de la Fase 3, porque es el mismo cambio transversal que vas a hacer vos.
- **El body como lista** está evaluado como "Sí" por Gaston y por vos (tu caso 12), y como "Parcial" por Javier (su 19). Los tres dicen lo mismo en el "por qué", así que conviene unificar la evaluación.

Algo que encontré de paso

El PR de Bruno (#8) ya está mergeado, y su `bruno.md` cita `anexo/ia/bruno_claude_01_diagnostico.pdf`, pero **ese PDF no está en `dev`**. Como el PDF lo coordinás vos, pedile que lo suba antes del tag `etapa-diagnostico`.

¿Querés que te arme el texto del review para el PR de Gaston?

Usuario (Cristian Morales) 02/11/2026 18:03

https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta
recopilando, revisa dev a ver si ya los 4 integramos nuestro diagnostico en dev

Claude 02/11/2026 18:04

▸ *Ejecutó un comando: Verificar qué diagnósticos están integrados en dev*

▸ *Ejecutó un comando: Verificar correcciones pendientes y PRs abiertos*

▸ *Ejecutó un comando: Verificar el conflicto de conftest.py entre los PRs #4 y #9*

Sí: **los cuatro diagnósticos ya están en `dev`**. Todavía no está el tag `etapa-diagnostico`: solo existe `estado-inicial`.

Integrante	PR	Ficha y resultados	Evidencias	Uso de IA	Transcripción
Cristian	#3	✓	20	✓	✓ PDF
Bruno	#8	✓	35	✓	**✗ falta el PDF**
Javier	#7	✓	24	✓	✓ PDF
Gaston	#10	✓	21	⚠ tabla rota	⚠ solo un registro `.md`

Antes de poner el tag, quedan tres cosas

Ninguna de las correcciones que marqué en los PRs llegó a `dev`:

1. **Bruno:** su `bruno.md` cita `anexo/ia/bruno_claude_01_diagnostico.pdf`, pero el archivo no está en el repo.
2. **Gaston, `docs/uso-ia/gaston.md`:** a la segunda fila de la tabla le falta la columna de evidencia y el `|` final, y el `## Detalle` quedó pegado a la tabla. Además, el diagnóstico (grado Alto) no tiene evidencia: el registro `.md` solo cubre la plantilla y una consulta.
3. **Gaston, `gaston_resultados_fase1.md`:** la nota sobre las solicitudes 7 y 11 sigue cortando la tabla de datos iniciales.

Conviene corregirlo en un PR chico y **poner el tag después**, para que `etapa-diagnostico` marque el diagnóstico completo. Si el tag queda antes, esos arreglos quedan después de la etapa.

Cuando esté, el tag lo ponés vos, desde `dev` actualizado:

```
git checkout dev
git pull
git tag -a etapa-diagnostico -m "Fase 1: diagnostico de los cuatro integrantes"
git push origin etapa-diagnostico
```

Lo importante: hay dos `conftest.py` que chocan

El PR #9 de Bruno (sus tests de la Fase 2) trae **su propio `tests/conftest.py`**, y entra en conflicto con el de tu PR #4: los dos crean el mismo archivo. Además, el diseño es distinto:

	El tuyo (PR #4)	El de Bruno (PR #9)
Fixture	`client`	`cliente`
Base	una nueva **por test**	**una sola para toda la corrida**
500	`raise_server_exceptions=False` solo en el caso 10	`False` para todos
Limpieza	descarta la base de cada test	borra a mano las filas que inserta

Con el suyo, los tests comparten la base: lo que crea un test lo ve el siguiente. Tus tests de altas y duplicados dependen de arrancar limpios. Y con `raise_server_exceptions=False` para todos, un error inesperado se convierte en un 500 en vez de mostrar el traceback.

Esto hay que resolverlo entre ustedes dos **antes de mergear cualquiera de los dos PRs**. Una opción razonable es un solo `conftest.py` con tu fixture por test como base, más los fixtures que inserta Bruno para sus casos 17 y 18, adaptados a esa base. Bruno coordina los tests, así que la decisión final es de él, pero tenés buenos argumentos para proponerla.

También siguen abiertos el PR #6 (la plantilla de Gaston) y el #5. Fijate si todavía hacen falta o si se pueden cerrar.

Usuario (Cristian Morales) 02/11/2026 18:05

revisa solo lo que esta mergeado

Claude 02/11/2026 18:06

▸ *Ejecutó un comando: Revisar los diagnósticos integrados de Bruno y Javier*

▸ *Ejecutó un comando: Revisar los scripts y prioridades de Bruno*

Revisé solo lo que está en `dev`. Los cuatro diagnósticos están completos en lo esencial, y los cuatro trabajos son buenos. Lo que hay son errores de formato y de evidencia, más algunas inconsistencias de criterio entre ustedes que conviene unificar antes de armar la tabla grupal.

Por integrante

Cristian: sin observaciones.

Javier: entró tal cual estaba el PR, así que siguen pendientes los cuatro ajustes que te marqué:

- el caso 10 dice "Sí", pero no notifica y es evidencia del hallazgo 3: tendría que ser "Parcial";

- el caso 19 se justifica por el mensaje, cuando el problema es el formato `detail`;
- el caso 12c dice "no el mensaje" cuando lo que falla es el historial;
- el caso 1 ("Sí") figura como evidencia del hallazgo 1 sin aclarar que es un contraste.

Bruno: es el diagnóstico más fuerte: inyección SQL comprobada (9a/9b), dos 500 en el reporte y un hallazgo propio, el 2b. Tiene cinco puntos:

1. **Falta el PDF** `anexo/ia/bruno_claude_01_diagnostico.pdf`, que su `bruno.md` cita.
2. **La prioridad está invertida respecto del criterio del grupo.** Los hallazgos 3 y 4 (código duplicado e índices) están en **Alta** sin casos, mientras que el 9, el 10 y el 11, con defectos comprobados, están en **Media**. Además, el 4 es el mismo hallazgo que tu 8 y que el 9 de Javier, que los dos pusieron en Media.
3. **El hallazgo 11 cita el caso 4 como evidencia**, pero el caso 4 está evaluado como "Sí": la lista vacía es correcta ahí. Es un contraste, no una evidencia.
4. **Los casos 13 y 14 se contradicen:** el id `0` da "Sí" con 404, pero el `-5` da "Parcial" porque "un id imposible debería ser 422". El `0` también es un id imposible.
5. **La ruta de los scripts no coincide:** el documento dice que `preparar_caso17.py` y `preparar_caso18.py` están "en esta carpeta" (`docs/diagnostico/`), pero están en `docs/evidencias/fase1/bruno/`.

Gaston:

1. **`docs/uso-ia/gaston.md` está roto:** a la segunda fila le falta la columna de evidencia y el `|` final, y `## Detalle` quedó pegado.
2. **El diagnóstico, que tiene intervención Alta, no tiene evidencia de IA.** El registro `.md` cubre solo la plantilla y una consulta.
3. **La nota sobre las solicitudes 7 y 11 corta la tabla** de datos iniciales en `gaston_resultados_fase1.md`.

Criterios que el grupo tiene que unificar

Esto va a aparecer en la tabla grupal y en la defensa, así que conviene acordarlo ahora:

Tema	Cómo está hoy
Recurso del body que no existe	Gaston: **422** (estudiante). Javier: **404** (responsable)
Body como lista (`[1, 2]`)	Vos y Gaston: **"Sí"**. Javier: **"Parcial"**. Los tres dan la misma razón
Qué es prioridad Alta	Vos, Javier y Gaston: defecto comprobado. Bruno: también incluye smells sin falla comprobada
Índices de columnas	Media para vos y Javier, Alta para Bruno

Para el tag

Los tres cambios que faltan para que el diagnóstico quede completo son el PDF de Bruno, la tabla rota de `gaston.md` y la evidencia de IA de Gaston. El resto (los ajustes de Javier y de Bruno, y los criterios) se puede corregir en la tabla grupal, sin bloquear el tag.

Usuario (Cristian Morales) 02/11/2026 18:06

bien quiero hacer el pr de dev a main con el tag correspondiente, estimo que es estado-diagnostico

Claude 02/11/2026 18:06

Antes de armarlo, dos aclaraciones:

1. El tag se llama `etapa-diagnostico`, no `estado-diagnostico`. Así lo listaste al principio: `estado-inicial`, `etapa-diagnostico`, `etapa-refactoring`, `etapa-pruebas`, `etapa-docs`, `estadorefactorizado`. Usá exactamente ese nombre, porque es el que va a figurar en el informe.

2. Según lo que acordaron, `dev` pasa a `main` recién al final, con el tag `estadorefactorizado`. Los tags de etapa se ponen sobre `dev`: no hace falta un PR a `main` para marcar el diagnóstico. Si ahora

mergeás `dev` a `main`, cambiás ese acuerdo. No está mal si el grupo lo decide, pero tiene que ser una decisión del grupo, y en el informe hay que explicar por qué `main` avanzó por etapas.

Lo que te recomiendo

Poner el tag en `dev`, sin PR a `main`:

```
git checkout dev
git pull
git log --oneline -1
git tag -a etapa-diagnostico -m "Fase 1: diagnostico de los cuatro integrantes"
git push origin etapa-diagnostico
```

El `git log` tiene que mostrar `45206f9 Merge pull request #10...` o un commit posterior, si antes mergean las correcciones.

Antes de taguear, decidí si esperás las tres correcciones (el PDF de Bruno, la tabla de `gaston.md` y la evidencia de IA de Gaston). Si taguéas ahora, esas correcciones van a quedar después de la etapa. Se puede, pero conviene aclararlo en el informe.

Si el grupo decide pasar a `main` por etapas

En ese caso:

1. Abrí el PR `dev` → `main` desde GitHub, con un título como "*Fase 1: diagnóstico*", y esperá la aprobación (`main` está protegida).
2. Mergealo con **merge commit**, no con squash, para que `main` conserve los commits de cada uno.
3. Poné el tag sobre el commit de merge en `main`:

`powershell git checkout main git pull git tag -a etapa-diagnostico -m "Fase 1: diagnostico de los cuatro integrantes" git push origin etapa-diagnostico`

¿Cuál de los dos caminos acordaron?

Usuario (Cristian Morales) 02/11/2026 18:07

lo que habiamos decidido era en main dejar todas las etapas claras pasando desde dev

Claude 02/11/2026 18:07

Perfecto, entonces vas por el segundo camino: un PR de `dev` a `main` por cada etapa, y el tag sobre `main`.

1. Antes de abrir el PR

Decidí si esperás las tres correcciones (el PDF de Bruno, la tabla de `gaston.md` y la evidencia de IA de Gaston). Como cada etapa queda marcada en `main`, conviene que lo que entre esté completo. Si no querés esperar, anotalo en la descripción del PR.

Confirmá también que en `dev` no haya nada de la Fase 2. Hoy está bien: tu PR #4 y el #9 de Bruno siguen abiertos. **No mergees ninguno de los dos a `dev` hasta que este PR esté mergeado en `main`.** Si entra antes, la etapa de diagnóstico en `main` va a incluir tests.

2. El PR

En GitHub: **Pull requests → New pull request**, con base `main` y compare `dev`.

Título: `Fase 1: diagnóstico`

Descripción:

```
## Etapa: diagnóstico (Fase 1)

Integra en `main` el diagnóstico de calidad externa e interna de los cuatro integrantes.
Al mergear se marca con el tag `etapa-diagnostico`.

### Contenido
```

```
| Integrante | Alcance (`main.py`) | PR |
|---|---|---|
| Cristian Morales | Conexión, tablas, datos de prueba, estudiantes y responsables (1–111) | #3
|
| Gaston Paschetta | Registrar solicitud (114–198) | #10 |
| Bruno Maza | Búsqueda, detalle y reporte (201–248, 321–361) | #8 |
| Javier Ovelar | Cambio de estado y asignación (251–318) | #7 |

Cada integrante aporta:
- `docs/diagnostico/<nombre>_resultados_fase1.md`: casos ejecutados y resultados observados
- `docs/diagnostico/<nombre>_ficha_smells.md`: hallazgos con impacto, riesgo, prioridad y
principio
- `docs/evidencias/fase1/<nombre>/`: capturas de cada caso
- `docs/uso-ia/<nombre>.md` y `anexo/ia/`: registro de uso de IA

### Fuera de esta etapa
- Tests (Fase 2): PR #4 y #9, se integran después de este merge.
- `main.py` no cambia: el sistema sigue en `estado-inicial`.
```

Si no esperás las correcciones, agregá al final:

```
### Pendiente
- PDF de uso de IA de Bruno (`anexo/ia/bruno_claude_01_diagnostico.pdf`)
- Tabla de `docs/uso-ia/gaston.md` y evidencia de IA del diagnóstico de Gaston
```

3. El merge

- Esperá la aprobación (`main` está protegida).
- Elegí **"Create a merge commit"**, no squash ni rebase. Así `main` conserva los commits de cada integrante.

4. El tag, sobre `main`

```
git checkout main
git pull
git log --oneline -1
git tag -a etapa-diagnostico -m "Fase 1: diagnostico de los cuatro integrantes"
git push origin etapa-diagnostico
```

El `git log` tiene que mostrar el merge del PR que acabás de cerrar.

Después, en el grupo, avisá que ya se pueden mergear a `dev` los PRs de la Fase 2. Primero tienen que resolver con Bruno el choque de los dos `conftest.py`.

Usuario (Cristian Morales) 02/11/2026 18:10

https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta/pull/11

una vez que mergeo este pr, ahi voy a la consola y le agrego el tag no?

Claude 02/11/2026 18:10

▸ *Ejecutó un comando: Verificar el PR #11 contra dev y main*

▸ *Ejecutó un comando: Verificar las referencias del PR #11, main y dev*

Sí, ese es el orden. Antes revisé el PR #11 y está bien armado:

- Apunta a `dev` tal como está hoy (`45206f9`, el merge del diagnóstico de Gaston), con los cuatro diagnósticos.
- No incluye tests ni cambios de la Fase 2.
- GitHub ya generó el merge de prueba, así que no tiene conflictos con `main`.

Los pasos:

1. **Mergealo con "Create a merge commit"**, no squash ni rebase. Así `main` conserva el historial de cada uno.
2. **En la consola**, primero traé `main` actualizado. Si no, el tag va a quedar sobre el `main` viejo:

`powershell git checkout main git pull git log --oneline -1`

El `git log` tiene que mostrar `Merge pull request #11...`. Si muestra otra cosa, no sigas. 3. **Creá y subí el tag:**

`powershell git tag -a etapa-diagnostico -m "Fase 1: diagnostico de los cuatro integrantes" git push origin etapa-diagnostico`

4. **Verificá:**

`powershell git tag git ls-remote --tags origin`

Tienen que aparecer `estado-inicial` y `etapa-diagnostico`.

Si querés, pasame la salida del último paso y lo confirmo desde acá.

Usuario (Cristian Morales) 02/11/2026 18:24

https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta/pull/9

ahi mergee lo mio, en base a lo que esta en este PR, si tomamos de base lo que ya esta en dev, que tiene que ajustar

Claude 02/11/2026 18:25

▸ *Ejecutó un comando: Leer los tests del PR #9 de Bruno*

▸ *Ejecutó un comando: Probar los tests de Bruno sobre dev con el conftest actual*

▸ *Ejecutó un comando: Validar los ajustes propuestos para el conftest con los tests de Bruno*

▸ *Ejecutó un comando: Guardar la prueba y limpiar la copia local*

▸ *Ejecutó un comando: Limpiar la copia local*

▸ *Ejecutó un comando: Limpiar la copia local*

Lo probé directamente: tomé `dev` (que ya tiene tus tests mergeados), agregué los tests de Bruno y apliqué los ajustes de abajo. Con eso da **38 passed**: tus 13 y sus 25. No subí nada: la prueba fue en mi copia local y ya la descarté.

El problema

El PR #9 crea su propio `tests/conftest.py`, y `dev` ya tiene el tuyo, así que GitHub va a marcar un conflicto en ese archivo. Además, sus tests usan otro fixture (`cliente`, con una sola base para toda la corrida), así que no corren con el tuyo así nomás. Lo probé: falla 1 test y dan error otros 2.

Lo que tiene que ajustar Bruno

1. Traer `dev` a su rama y quedarse con el `conftest.py` de `dev`. Su conftest se descarta. Las partes que necesita las agrega al de `dev` (puntos 3 y 4). El `sys.path.insert` tampoco hace falta, porque lo resuelve el `pytest.ini`.

2. Cambiar `cliente` por `client` en `test_buscar.py`, `test_detalle.py` y `test_reporte.py`. Es un reemplazo directo y no cambia nada más.

3. Usar un cliente que no corte el test con los 500. Sus casos 10, 17 y 18 esperan un 500. Con `client` reciben la excepción, como te pasó con tu caso 10. Ahora que son 4 usos (los 3 de él más el tuyo), conviene llevarlo al `conftest.py`:

```
@pytest.fixture
def client_sin_excepciones(client):
    """Mismo sistema y misma base que client, pero un error interno
    devuelve 500 en lugar de cortar el test."""
```

```
    with TestClient(client.app, raise_server_exceptions=False) as c:
        yield c
```

Esos tres tests pasan a pedir `client_sin_excepciones` en vez de `client`. El resto sigue con `client`, así un error inesperado muestra el traceback.

4. Adaptar los fixtures de fechas inválidas. Tienen que depender de `client`, para insertar en la base de ese test, y tomar `main` de `sys.modules["main"]`. Como cada test arranca con su propia base, **ya no hace falta borrar nada**: `_borrar_solicitudes_de_prueba` sobra.

5. Reescribir o sacar `test_el_reporte_se_recupera_al_quitar_el_dato_malo`. Depende de que corra después de los otros dos. Con una base por test, siempre pasa y no prueba nada. Si quiere conservar la idea, que lo haga en un solo test: insertar el dato malo, verificar el 500, borrarlo y verificar el 200.

6. Rehacer la captura de la suite completa. La de su PR muestra solo sus tests con su conftest. La nueva tiene que mostrar los 38.

Para vos

- **Sacá el comentario "PROVISORIO"** del `conftest.py` cuando Bruno lo integre: ya deja de ser provisorio.
- **Opcional:** tu caso 10 puede pasar a usar `client_sin_excepciones` en vez de crear su propio cliente. Queda más corto y consistente.
- **Para unificar en el grupo:** Bruno marca los defectos con `DEFECTO` en el nombre y vos con `acepta_...` más el comentario actual/esperado. Las dos formas sirven, pero en el informe conviene explicar el criterio o acordar uno.

Si te sirve para pasárselo a Bruno, guardé el diff de la prueba que funcionó.

Usuario (Cristian Morales) 06/11/2026 10:17

bien retomemos, revisa el repo dev como esta

https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta

y ver si puedo continuar con mi parte

Claude 06/11/2026 10:17

▸ *Ejecutó un comando: Revisar dev, main, tags y PRs abiertos*

▸ *Ejecutó un comando: Correr la suite de dev y revisar el plan*

▸ *Ejecutó un comando: Leer el plan del README y revisar el repositorio*

Revisé `dev`. Ya podés arrancar con tu parte de la Fase 3, pero con una condición: el merge espera a los tests de Gaston.

Cómo está `dev`

- **Tests de la Fase 2:** ya están los tuyos (#4), los de Javier (#12) y los de Bruno (#9). Los corrí desde GitHub y dan **47 passed** (13 tuyos, 9 de Javier y 25 de Bruno), todos con tu `conftest.py` y el `client_sin_excepciones` que habíamos armado.
- **`main.py` sigue igual que en `estado-inicial`.** Nadie empezó a refactorizar todavía.
- **El tag `etapa-diagnostico`** está sobre `main`.

Lo que falta

1. **Los tests de Gaston** (`POST /solicitudes`). Son los únicos que no hay. Es el pendiente más importante, por lo que te explico abajo.
2. **El `conftest.py` todavía dice "PROVISORIO"** arriba, aunque ya lo usan los cuatro. Sacalo en tu primer PR de refactor.
3. **Siguen pendientes de la Fase 1:** el PDF de IA de Bruno y la tabla rota de `docs/uso-ia/gaston.md`.
4. **El README dice que los tags de etapa van sobre `dev`**, pero ustedes decidieron pasarlos a `main`. Hay que actualizarlo para que coincida con lo que hacen. Javier coordina `/docs`, así que

puede encargarse él.
5. **Los PRs #5 y #6** siguen abiertos desde octubre. Si ya no hacen falta, conviene cerrarlos.

Tu parte: los transversales 1 y 2

Tus hallazgos 1 (Global State) y 2 (efectos secundarios al importar) cambian la conexión y el arranque. Eso afecta a **todo** `main.py`, incluido `registrar`, que es la parte de Gaston. Por eso:

- **Podés empezar ya** en una rama `refactor/conexion` desde `dev`.
- **No mergees hasta que estén los tests de Gaston en `dev`.** Sin ellos, si rompés algo en el alta de solicitudes, nada lo detecta.
- **Hacelo antes que los demás empiecen sus refactors**, como estaba planeado. Si tocan `main.py` en paralelo, van a tener conflictos en todo el archivo.

Antes de escribir código, pensá estas tres preguntas. Pasame las respuestas y armamos el plan juntos:

1. **¿Qué debería pasar al hacer `import main`?** Hoy abre la base, crea las tablas y carga los datos de prueba. ¿Qué de eso debería pasar al importar, qué al arrancar la app y qué nunca automáticamente?
2. **¿Cómo le llega la conexión a cada endpoint** si dejan de existir las variables globales `conn` y `c`? FastAPI tiene un mecanismo pensado para esto. Buscá `Depends`.
3. **¿Qué pasa con tu `conftest.py`?** Hoy funciona gracias al efecto secundario: cambia de carpeta y reimporta `main`. Si resolvés el hallazgo 2, ese truco deja de hacer falta. ¿Cómo le dirías al test qué base usar?

Un recordatorio para cuando refactorices: los 47 tests tienen que seguir en verde **sin cambiar ningún assert**. Lo único que puede cambiar es el `conftest.py`, porque es infraestructura y no comportamiento. Si algún assert tiene que cambiar, ya no es refactoring.

Usuario (Cristian Morales) 06/11/2026 10:18

en que consiste la fase 3?

Claude 06/11/2026 10:18

▸ *Ejecutó un comando: Buscar la definición de la Fase 3 en la preparación*

▸ *Ejecutó un comando: Leer la consigna de la Fase 3*

La Fase 3 es el **refactoring**: mejorar la estructura interna de `main.py` sin cambiar lo que hace el sistema. Los tests de la Fase 2 sirven como prueba: tienen que dar verde antes y después de cada cambio.

Qué pide la consigna

- **Al menos 4 refactorings**, y tres son obligatorios:
- **Extract Method** (lo garantiza Bruno);
- **Move Method**, o reubicar una responsabilidad (lo garantiza Javier);
- **Introduce Parameter Object**, o una simplificación de parámetros equivalente (lo garantiza Gaston).
- **Cada refactoring con su código antes y después, y su propio commit.** Por eso los PRs se mergean con merge commit y no con squash: si los commits se aplastan, se pierde la evidencia.
- **Vincular cada mejora con un principio** (SRP, OCP, DIP, nombres, manejo de errores, acoplamiento). Es el criterio "SOLID y Clean Code", que vale 20%. El "refactoring seguro" vale otro 20%.
- **Cada refactoring tiene que salir de un hallazgo de la tabla de smells.** Primero el problema, después la solución.

Cómo se trabaja

1. **Antes de empezar, acuerden la estructura** de carpetas y módulos a la que quieren llegar, y déjenla escrita en `docs/` como decisión técnica. Tiene que justificarse con los smells del diagnóstico. Este paso todavía no está hecho.

2. **Primero los cambios transversales, que hace uno solo.** Son los tuyos: sacar la conexión global (hallazgo 1) y los efectos secundarios al importar (hallazgo 2). Se mergean, y recién después arranca el resto. Si no, los cuatro tocan el mismo archivo y chocan en cada merge.
3. **Después, cada uno trabaja su parte** en una rama `refactor/<tema>`, con un PR que revisa otro compañero. Eso también cubre la etapa de code review.
4. **Al cerrar, se pone el tag `etapa-refactoring`.**

La regla que define si es refactoring

Los 47 tests tienen que seguir en verde **sin cambiar ningún assert**. Si un test tiene que cambiar su expectativa (por ejemplo, de `200` a `422`), ya no es refactoring: es **corrección de defecto**. Se puede hacer, pero en un commit aparte, justificado con el hallazgo y explicado así en el informe. Es lo que dejaste anotado con el comentario "actual/esperado" en tus tests.

Antes de que arranques

Faltan dos cosas que la Fase 3 necesita:

- **Los tests de Gaston.** Sin ellos, el alta de solicitudes no tiene red de seguridad.
- **La decisión de estructura en `docs/`.** Si querés, la armamos a partir de las cuatro fichas de smells: de ahí sale qué módulos tienen sentido (conexión, repositorios, servicios, validación) y por qué.

¿Arrancamos por la propuesta de estructura o por tus transversales?

Usuario (Cristian Morales) 06/11/2026 10:21

bien armemos la estuctura y transvesales , solamente asi podemos liberar eso y el equipo puede continuar cada uno, despues hacemos el refactor grueso de lo que me toca

Claude 06/11/2026 10:22

Arranco por la estructura: la armo a partir de los hallazgos de las cuatro fichas, para que cada módulo quede justificado.

▸ *Ejecutó un comando: Extraer los hallazgos de las cuatro fichas de smells*

Revisé las cuatro fichas. Esta es la propuesta, con cada módulo justificado por los hallazgos.

Estructura propuesta

```
main.py                 crea la app y registra los routers (uvicorn main:app sigue igual)
app/
  db.py                 conexión por request (Depends), creación de tablas, sqlite3.Row
  seed.py               datos de prueba, cargados de forma explícita
  dominio.py            tipos, áreas, estados, transiciones y reglas por tipo
  schemas.py            modelos Pydantic de entrada
  routers/
    estudiantes.py      Cristian   (estudiantes)
    responsables.py     Cristian   (responsables)
    solicitudes.py      Gaston     (registrar)
    estados.py          Javier     (cambio de estado y asignación)
    consultas.py        Bruno      (búsqueda, detalle y reporte)
  servicios/            se crean en la fase 3 de cada uno, si hacen falta
    notificaciones.py   historial y notificación (Gaston + Javier)
```

Módulo	Hallazgos que lo justifican
`db.py`	Global State (C1), efectos al importar (C2), índices de columnas (C8, J9, B4, G8), conversión de filas duplicada (C9, B3)
`seed.py`	efectos al importar (C2), datos de prueba duplicados (C10)
`dominio.py`	áreas como strings (C6, C13), estados y transiciones (J2, J3, J10), reglas por tipo (G2, G6), constantes del reporte (B8)
`schemas.py`	falta de modelo de entrada (C5, J5, G5), validación de email duplicada (C4)

Módulo	Hallazgos que lo justifican
`routers/`	SRP: cada archivo tiene una sola área y un solo dueño, así que pueden trabajar en paralelo sin conflictos
`notificaciones.py`	historial y notificación duplicados (J8, G3), excepción silenciada (J7, G7)

Qué hacés vos ahora (lo transversal) y qué no

Ahora, para liberar al equipo:

1. Crear `app/db.py` y `app/seed.py`, y sacar `conn`, `c` y la carga de datos del import (tus hallazgos 1 y 2).
2. Mover cada endpoint, **tal cual está**, a su router. Es un movimiento puro: sin cambiar ni una línea de lógica. Después cada uno trabaja en su propio archivo.
3. Adaptar el `conftest.py` (y los fixtures de Bruno que usan `main.c`) a la nueva forma de obtener la base.

No ahora:

- `dominio.py`, `schemas.py` y `notificaciones.py` los crea cada dueño cuando le toque. Solo los dejamos definidos en la decisión técnica.
- **Pasar los errores de 200 a 422** (C3, J1, G4) no es refactoring: cambia el comportamiento y cambia asserts. Va después, como corrección de defecto acordada por el grupo, con el criterio de códigos ya unificado.

Tres decisiones antes de escribir código

1. **¿Movemos los endpoints a routers ahora o los dejamos en `main.py`?** Recomiendo moverlos ahora, para que nadie choque. El costo: ese movimiento es un "Move", y Javier garantiza Move Method. A él le queda igual el suyo (las transiciones a `dominio.py` o la notificación a un servicio), pero conviene avisarle.
2. **¿Cómo le dice el test a la app qué base usar?** Hay dos opciones: - **A.** `app.dependency_overrides[get_db]`: el test reemplaza la dependencia por una base temporal. Es el mecanismo estándar de FastAPI. - **B.** Una variable de entorno con la ruta de la base, que el `conftest.py` setea por test. Es más simple, pero menos explícita.
3. **¿Dónde se cargan los datos de prueba?** Las opciones: en el arranque de la app (`lifespan`), solo si la base está vacía como hoy, o solo cuando se pide explícitamente (por ejemplo `run.bat reset`). Ojo: los tests de Bruno cuentan con las 6 solicitudes de prueba, así que el `conftest.py` las tiene que cargar.

Decime qué elegís en cada una, o si querés ajustar la estructura, y armamos el primer paso: `db.py`.

Usuario (Cristian Morales) 06/11/2026 10:24

hagamos para liberar el equipo

Claude 06/11/2026 10:28

▸ *Ejecutó un comando: Crear una rama local de referencia y revisar los tests*

▸ *Ejecutó un comando: Revisar test_reporte.py y los scripts de arranque*

▸ *Ejecutó un comando: Ubicar las tablas y los datos de prueba en main.py*

▸ *Ejecutó un comando: Extraer tablas y datos de prueba a app/ (copia local)*

▸ *Ejecutó un comando: Ordenar imports y registrar el paso 1 (copia local)*

▸ *Ejecutó un comando: Reemplazar la conexión global por una por request (copia local)*

▸ *Ejecutó un comando: Verificar que no queden globales*

▸ *Ejecutó un comando: Adaptar conftest.py y correr los tests*

▸ *Ejecutó un comando: Verificar que importar main no crea la base y que el servidor funciona*

▸ *Ejecutó un comando: Registrar el paso 2 (copia local)*

▸ *Ejecutó un comando: Mover los endpoints a routers y correr los tests (copia local)*

▸ *Ejecutó un comando: Verificar que las funciones movidas no cambiaron*

▸ *Ejecutó un comando: Comparar el código movido línea por línea*

▸ *Ejecutó un comando: Registrar el paso 3 (copia local)*

▸ *Ejecutó un comando: Redactar la decisión técnica (copia local)*

▸ *Ejecutó un comando: Exportar los cambios como patches y verificar que se aplican*

▸ *Ejecutó un comando: Verificar los patches sobre dev*

▸ *Envió el archivo refactor-conexion.zip*

Armé la estructura y los cambios transversales en una rama `refactor/conexion`, en 4 commits. Los 47 tests pasan después de cada uno, sin cambiar ningún assert. No subí nada: te los paso para que los apliques vos.

Qué hace cada commit

#	Commit	Qué cambia
1	`refactor: extraer la creacion de tablas...`	Las tablas pasan a `app/db.py` y los datos de prueba a `app/seed.py`. `main.py` los sigue llamando igual.
2	`refactor: reemplazar la conexion global...`	Se eliminan `conn` y `c` globales. Cada request recibe su conexión con `Depends(get_db)`. `crear_app(ruta_db)` arma la app y la base se prepara al **arrancar**, no al importar. El `conftest.py` crea una app por test, sin reimportar `main`. Resuelve tus hallazgos **1 y 2**.
3	`refactor: mover los endpoints...`	Cada endpoint pasa, **sin tocar su lógica**, a su router en `app/routers/`. Verifiqué que las líneas sean exactamente las mismas: solo cambian de archivo.
4	`docs: decision tecnica...`	`docs/decisiones/01_estructura_y_conexion.md`: problema, estructura, módulos previstos con sus hallazgos, alternativas descartadas y cómo se verificó.

Además, verifiqué que `import main` en una carpeta vacía no crea ningún archivo, y que `uvicorn main:app` levanta el sistema con los datos de prueba y responde igual que antes.

Cómo aplicarlo

Descomprimí `refactor-conexion.zip` en una carpeta (por ejemplo `C:\temp\refactor-conexion`) y:

```
git checkout dev
git pull
git checkout -b refactor/conexion
git am C:\temp\refactor-conexion\*.patch
.\run.bat test
git push -u origin refactor/conexion
```

Tiene que dar `47 passed`. Los commits quedan a tu nombre, con Claude como coautor en cada mensaje, así queda registrado el uso de IA.

Antes de mergear

- **Esperá los tests de Gaston**, como hablamos. Como prueban la API desde afuera, deberían funcionar con la nueva estructura sin cambios. Si ya los escribió usando `main.c` o `main.conn`, va a tener que pasar a `conexion_db`.
- **Avisale a Bruno:** en `test_reporte.py` cambié cómo se borra el dato malo (ahora usa `conexion_db` en vez de `main.c`), y sus fixtures de fechas inválidas pasaron al nuevo esquema. Ningún assert cambió.
- **Que lo revise alguien que conozca el código**, idealmente Javier, que garantiza Move Method. El commit 3 es un movimiento, pero el suyo sigue disponible: mover las transiciones a `dominio.py` o la notificación a un servicio.
- **Mergeá con "Create a merge commit"**, para que queden los 4 commits como evidencia.

Después del merge

Cada uno trabaja en su router, sin pisarse:

- Gaston en `app/routers/solicitudes.py`;
- Javier en `app/routers/estados.py`;

- Bruno en `app/routers/consultas.py`;
- vos en `estudiantes.py` y `responsables.py`, con el refactor grueso de tu parte.

Esta vez el código lo escribí yo, así que en tu `docs/uso-ia/cristian.md` va a hacer falta una fila de la Fase 3 que lo diga, igual que hicimos con la Fase 2. Cuando tengas el PR abierto, si querés, lo reviso.

Usuario (Cristian Morales) 06/11/2026 11:12

pasame el paso a paso de lo que tengo que modificar, con codigo si hay

Claude 06/11/2026 11:12

▸ *Ejecutó un comando: Preparar el paso a paso manual*

Va en el mismo orden que los 4 commits. Después de cada paso corré `.\run.bat test`: tiene que dar **47 passed** antes de commitear.

Arrancá desde `dev` actualizado:

```
git checkout dev
git pull
git checkout -b refactor/conexion
```

Paso 1: tablas y datos de prueba a `app/`

1.1 Creá la carpeta `app/` y adentro un archivo vacío `__init__.py`.

1.2 Creá `app/db.py`:

```
"""Acceso a la base de datos: conexión y creación de las tablas."""

import sqlite3

RUTA_DB_POR_DEFECTO = "solicitudes.db"


def conectar(ruta_db):
    """Abre una conexión a la base indicada."""
    return sqlite3.connect(ruta_db, check_same_thread=False)


def crear_tablas(conn):
    """Crea las tablas del sistema si todavía no existen."""
    c = conn.cursor()
    # PEGAR ACÁ los 4 c.execute("""CREATE TABLE ...""") de main.py (líneas 10–18),
    # con 4 espacios más de sangría
    conn.commit()
```

1.3 Creá `app/seed.py`:

```
"""Datos de prueba sintéticos del sistema."""


def base_vacia(conn):
    """Indica si la base todavía no tiene estudiantes cargados."""
    c = conn.cursor()
    c.execute("SELECT COUNT(*) FROM estudiantes")
    return c.fetchone()[0] == 0


def cargar_datos_de_prueba(conn):
    """Carga los estudiantes, responsables, solicitudes e historial de prueba."""
    c = conn.cursor()
    # PEGAR ACÁ los c.execute("INSERT ...") de main.py (líneas 24–53),
```

```
    # con la misma sangría que ya tienen (4 espacios)
    conn.commit()
```

1.4 En `main.py`, reemplazá **desde la línea 1 hasta la 54** (imports, conexión, tablas y datos de prueba) por:

```
from fastapi import FastAPI, Body, HTTPException
from datetime import datetime

from app.db import conectar, crear_tablas, RUTA_DB_POR_DEFECTO
from app.seed import base_vacia, cargar_datos_de_prueba

app = FastAPI(title="Sistema de Gestion de Solicitudes Academicas")

conn = conectar(RUTA_DB_POR_DEFECTO)
c = conn.cursor()

crear_tablas(conn)
if base_vacia(conn):
    cargar_datos_de_prueba(conn)
```

Los endpoints quedan igual. Todavía usan `conn` y `c` globales: eso se resuelve en el paso 2.

1.5 Probá y commiteá:

```
.\run.bat test
git add main.py app/
git commit -m "refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y
app/seed.py"
```

Paso 2: conexión por request, en lugar de la global

2.1 En `app/db.py`, agregá el import de `Request` debajo de `import sqlite3`:

```
from fastapi import Request
```

Y al final del archivo:

```
def get_db(request: Request):
    """Dependencia de FastAPI: abre una conexión por request y la cierra al terminar."""
    conn = conectar(request.app.state.ruta_db)
    try:
        yield conn
    finally:
        conn.close()
```

2.2 En `main.py`, reemplazá el bloque que escribiste en 1.4 por:

```
import sqlite3
from contextlib import asynccontextmanager
from datetime import datetime

from fastapi import APIRouter, Body, Depends, FastAPI, HTTPException

from app.db import RUTA_DB_POR_DEFECTO, conectar, crear_tablas, get_db
from app.seed import base_vacia, cargar_datos_de_prueba

router = APIRouter()
```

Ya no existen `app`, `conn` ni `c` globales.

2.3 En **todos** los endpoints:

- cambié `@app.` por `@router.` (con buscar y reemplazar, son 10);
- agregá `conn: sqlite3.Connection = Depends(get_db)` como **último** parámetro;
- agregá `c = conn.cursor()` como **primera línea** del cuerpo.

Por ejemplo:

```
# antes
@app.post("/estudiantes")
def crear_estudiante(data: dict = Body(...)):
    if "nombre" not in data ...

# después
@router.post("/estudiantes")
def crear_estudiante(data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)):
    c = conn.cursor()
    if "nombre" not in data ...
```

Los endpoints sin parámetros quedan así: `def listar_estudiantes(conn: sqlite3.Connection = Depends(get_db)):`. En `buscar`, el parámetro va después de `orden: str = "fecha",`.

2.4 `guardar_solicitud` no es un endpoint, pero también usaba la conexión global. Recibe `conn` como **primer** parámetro:

```
def guardar_solicitud(conn, estudiante_id, tipo, descripcion, prioridad, estado, responsable_id,
area, fecha_alta, fecha_mod, canal, obs):
    c = conn.cursor()
    c.execute("""INSERT INTO solicitudes ...
```

Y en `registrar`, la llamada pasa a ser:

```
id = guardar_solicitud(conn, data["estudiante_id"], data["tipo"], ...
```

`calcular_prioridad` no se toca.

2.5 Al final de `main.py`, agregá:

```
def preparar_base(ruta_db):
    """Crea las tablas y, si la base está vacía, carga los datos de prueba."""
    conn = conectar(ruta_db)
    try:
        crear_tablas(conn)
        if base_vacia(conn):
            cargar_datos_de_prueba(conn)
    finally:
        conn.close()


def crear_app(ruta_db=RUTA_DB_POR_DEFECTO):
    """Crea la aplicación sobre la base indicada.

    La base se prepara al arrancar la aplicación, no al importar el módulo.
    """
    @asynccontextmanager
    async def lifespan(app):
        preparar_base(app.state.ruta_db)
        yield

    app = FastAPI(title="Sistema de Gestion de Solicitudes Academicas", lifespan=lifespan)
    app.state.ruta_db = ruta_db
    app.include_router(router)
    return app
```

```
app = crear_app()
```

2.6 Reemplazá `tests/conftest.py` completo por:

```
"""Configuración común de los tests.

Cada test usa una aplicación creada sobre su propia base temporal, cargada con
los datos de prueba al arrancar: 4 estudiantes, 5 responsables y 6 solicitudes.
"""

import pytest
from fastapi.testclient import TestClient

from app.db import conectar
from main import crear_app


@pytest.fixture
def client(tmp_path, monkeypatch):
    """Cliente sobre una aplicación con una base nueva para este test."""
    # notificaciones.log se escribe en la carpeta actual: que quede en la temporal
    monkeypatch.chdir(tmp_path)

    app = crear_app(tmp_path / "solicitudes_test.db")
    with TestClient(app) as test_client:
        yield test_client


@pytest.fixture
def client_sin_excepciones(client):
    """Mismo sistema y misma base que client, pero un error interno
    devuelve 500 en lugar de cortar el test."""
    with TestClient(client.app, raise_server_exceptions=False) as c:
        yield c


# --- datos que reproducen los defectos del reporte ---------------------------

@pytest.fixture
def conexion_db(client):
    """Conexión directa a la base del test, para preparar datos que la API no permite cargar."""
    conn = conectar(client.app.state.ruta_db)
    yield conn
    conn.close()


def _insertar_solicitud(conn, descripcion, fecha_alta):
    """Inserta una solicitud en la base del test en curso."""
    conn.execute(
        "INSERT INTO solicitudes "
        "(estudiante_id, tipo, descripcion, prioridad, estado, responsable_id, "
        "area, fecha_alta, fecha_mod, canal, obs) "
        "VALUES (1, 'CERT', ?, 3, 'PEND', NULL, 'Alumnado', ?, ?, 'WEB', '')",
        (descripcion, fecha_alta, fecha_alta),
    )
    conn.commit()


@pytest.fixture
def solicitud_con_fecha_invalida(conexion_db):
    """Solicitud con fecha_alta en un formato que el reporte no interpreta.
    Reproduce el caso 17 del diagnostico."""
```

```
    _insertar_solicitud(conexion_db, "Test - fecha sin hora", "2026-11-01")


@pytest.fixture
def solicitud_con_fecha_nula(conexion_db):
    """Solicitud con fecha_alta nula. Reproduce el caso 18."""
    _insertar_solicitud(conexion_db, "Test - fecha nula", None)
```

2.7 En `tests/test_reporte.py`, en `test_DEFECTO_el_reporte_se_recupera_al_quitar_el_dato_malo`:

- agregá `conexion_db` a los parámetros:

`python def test_DEFECTO_el_reporte_se_recupera_al_quitar_el_dato_malo( client_sin_excepciones, solicitud_con_fecha_invalida, conexion_db):`

- sacá el `import sys` y reemplazá las tres líneas de `main = sys.modules[...]`, `main.c.execute(...)` y `main.conn.commit()` por:

`python conexion_db.execute("DELETE FROM solicitudes WHERE descripcion LIKE 'Test - %'") conexion_db.commit()`

Los dos `assert` quedan iguales.

2.8 Probá. Además, comprobá que importar ya no crea la base:

```
.\run.bat test
cd $env:TEMP
python -c "import sys; sys.path.insert(0, r'C:\ruta\a\tu\repo'); import main"
Get-ChildItem solicitudes.db    # no tiene que existir
cd C:\ruta\a\tu\repo
```

Commiteá:

```
git add main.py app/ tests/conftest.py tests/test_reporte.py
git commit -m "refactor: reemplazar la conexion global por una conexion por request"
```

Paso 3: un router por área

3.1 Creá la carpeta `app/routers/` con un `__init__.py` vacío.

3.2 Creá estos 5 archivos. Cada uno empieza con este encabezado y después lleva sus funciones, **cortadas de `main.py` tal cual están**:

```
"""<descripción>"""

import sqlite3
from datetime import datetime            # solo si el archivo usa datetime

from fastapi import APIRouter, Body, Depends, HTTPException   # solo lo que use

from app.db import get_db

router = APIRouter()
```

Archivo	Descripción	Funciones	Imports de FastAPI	datetime
`estudiantes.py`	Estudiantes: alta y listado.	`crear_estudiante`, `listar_estudiantes`	`APIRouter, Body, Depends`	no
`responsables.py`	Responsables: alta y listado.	`nuevoResponsable`, `get_resp`	`APIRouter, Body, Depends`	no
`solicitudes.py`	Solicitudes: registro de una solicitud nueva.	`guardar_solicitud`, `calcular_prioridad`, `registrar`	`APIRouter, Body, Depends`	sí

Archivo	Descripción	Funciones	Imports de FastAPI	datetime
`estados.py`	Solicitudes: cambio de estado y asignación de responsable.	`proc_estado`, `asignar`	`APIRouter, Body, Depends, HTTPException`	sí
`consultas.py`	Solicitudes: búsqueda, detalle y reporte.	`buscar`, `getSolicitud`, `reporte`	`APIRouter, Depends, HTTPException`	sí

Al moverlas no cambies **ninguna** línea de las funciones: solo cambian de archivo.

3.3 `main.py` queda solo con esto:

```
"""Punto de entrada: crea la aplicación y registra los routers."""

from contextlib import asynccontextmanager

from fastapi import FastAPI

from app.db import RUTA_DB_POR_DEFECTO, conectar, crear_tablas
from app.routers import consultas, estados, estudiantes, responsables, solicitudes
from app.seed import base_vacia, cargar_datos_de_prueba


def preparar_base(ruta_db):
    # igual que en el paso 2.5


def crear_app(ruta_db=RUTA_DB_POR_DEFECTO):
    # igual que en el paso 2.5, salvo que app.include_router(router)
    # se reemplaza por:
    app.include_router(estudiantes.router)
    app.include_router(responsables.router)
    app.include_router(solicitudes.router)
    app.include_router(estados.router)
    app.include_router(consultas.router)
    return app


app = crear_app()
```

3.4 Probá y commiteá:

```
.\run.bat test
git add main.py app/
git commit -m "refactor: mover los endpoints de main.py a un router por area"
```

Paso 4: la decisión técnica

Creá `docs/decisiones/01_estructura_y_conexion.md`. El contenido completo está en el `.patch` 0004 del zip que te mandé: abrilo con un editor de texto y copiá las líneas que empiezan con `+`, sin el `+`. Si preferís, te paso el `.md` suelto.

```
git add docs/decisiones/
git commit -m "docs: decision tecnica de estructura del proyecto y conexion a la base"
git push -u origin refactor/conexion
```

Verificación final

- `.\run.bat test`: **47 passed**.
- `.\run.bat` levanta el sistema, y en `/docs` están los 10 endpoints, que responden igual que antes.
- `git log --oneline dev..refactor/conexion` muestra los 4 commits.

Si en algún paso no da 47, pasame el error antes de seguir. Lo más probable es un import que falta en un router o un `c = conn.cursor()` que quedó afuera.

No voy a hacer push: me pediste que no lo hiciera y esta sesión tiene acceso de solo lectura al repo. La rama `refactor/conexion` es mi copia de referencia local. El push lo hacés vos desde tu máquina, siguiendo el paso a paso o con el zip de patches. Dejo mi copia local fuera de esa rama para que no vuelva a aparecer el aviso.

▸ *Ejecutó un comando: Dejar la copia local sin rama activa (sin push)*

Listo. Mi copia local quedó sin rama activa y sin cambios pendientes. No hice ningún push ni commit en tu repo. La rama `refactor/conexion` con los 4 commits la crea y la sube cada vez quien siga el paso a paso: vos.

Usuario (Cristian Morales) 06/11/2026 11:16

[Archivos adjuntos: salida de los tests]

Claude 06/11/2026 11:16

Todos los tests fallan por lo mismo antes de empezar: al preparar la base, `base_vacia()` hace `SELECT COUNT(*) FROM estudiantes` y la tabla no existe. Eso significa que **`crear_tablas()` no está creando las tablas**. Hay tres causas posibles, de la más probable a la menos:

1. `crear_tablas` quedó sin los `CREATE TABLE`. Si en `app/db.py` dejaste solo el comentario `# PEGAR ACÁ ...`, la función crea el cursor, hace `commit` y no crea nada. Tiene que quedar así:

```
def crear_tablas(conn):
    """Crea las tablas del sistema si todavía no existen."""
    c = conn.cursor()
    c.execute("""CREATE TABLE IF NOT EXISTS estudiantes (
        id INTEGER PRIMARY KEY AUTOINCREMENT, nombre TEXT, apellido TEXT, legajo TEXT, email
TEXT, carrera TEXT)""")
    c.execute("""CREATE TABLE IF NOT EXISTS responsables (
        id INTEGER PRIMARY KEY AUTOINCREMENT, nombre TEXT, area TEXT, email TEXT, activo
INTEGER)""")
    c.execute("""CREATE TABLE IF NOT EXISTS solicitudes (
        id INTEGER PRIMARY KEY AUTOINCREMENT, estudiante_id INTEGER, tipo TEXT, descripcion
TEXT, prioridad INTEGER,
        estado TEXT, responsable_id INTEGER, area TEXT, fecha_alta TEXT, fecha_mod TEXT, canal
TEXT, obs TEXT)""")
    c.execute("""CREATE TABLE IF NOT EXISTS historial (
        id INTEGER PRIMARY KEY AUTOINCREMENT, solicitud_id INTEGER, estado_ant TEXT,
estado_nuevo TEXT, fecha TEXT, comentario TEXT)""")
    conn.commit()
```

2. El orden está invertido. En `main.py` (paso 1) o en `preparar_base` (pasos 2 y 3), `crear_tablas(conn)` tiene que ir **antes** de `if base_vacia(conn):`.

3. `crear_tablas` existe pero no se llama. Fijate que esté la línea `crear_tablas(conn)` y no solo el import.

Para saber cuál es, mirá estas dos cosas:

```
Get-Content app\db.py
Select-String -Path main.py -Pattern "crear_tablas|base_vacia"
```

Si no lo encontrás, pasame la salida y decime en qué paso estás (1, 2 o 3).

Usuario (Cristian Morales) 06/11/2026 11:17

FAILED tests/test_asignacion.py::test_asignar_responsable_valido - AssertionError: assert {'error': 'so...ud no existe'} == {'id': 1, 're...tado': 'ASIG'}
FAILED tests/test_asignacion.py::test_asignar_responsable_inactivo - AssertionError: assert {'error':

'so...ud no existe'} == {'error': 're...ble inactivo'}
FAILED tests/test_asignacion.py::test_reasignar_no_registra_historial - AssertionError: assert {'error': 'so...ud no existe'} == {'id': 2, 're...tado': 'ASIG'}
FAILED tests/test_buscar.py::test_sin_filtros_devuelve_todas - assert 0 == 6
FAILED tests/test_buscar.py::test_filtro_por_estado - assert 0 == 2
FAILED tests/test_buscar.py::test_filtro_por_prioridad - assert 0 == 3
FAILED tests/test_buscar.py::test_DEFECTO_orden_invalido_se_ignora_en_silencio - assert 0 == 6
FAILED tests/test_buscar.py::test_DEFECTO_inyeccion_sql_evade_el_filtro - assert 0 == 1
FAILED tests/test_detalle.py::test_devuelve_la_solicitud_con_su_historial - assert 404 == 200
FAILED tests/test_estado.py::test_cambiar_estado_asig_a_en_proc - assert 404 == 200
FAILED tests/test_estado.py::test_cambiar_estado_transicion_no_permitida - assert 404 == 200
FAILED tests/test_estado.py::test_rechazar_acepta_motivo_nulo - assert 404 == 200
FAILED tests/test_estado.py::test_cambiar_estado_a_asig_sin_responsable - assert 404 == 200
FAILED tests/test_reporte.py::test_el_porcentaje_de_resueltas_es_coherente - ZeroDivisionError: division by zero

Claude 06/11/2026 11:17

Avanzaste: las tablas ya se crean. Ahora el problema es otro. **La tabla `solicitudes` está vacía.** Todo lo que falla busca solicitudes:

- `buscar` devuelve 0 en vez de 6;
- el detalle y el cambio de estado dan 404 ("no existe");
- `asignar` responde "solicitud no existe";
- el reporte divide por 0 porque el total es 0.

Los tests de estudiantes y responsables pasan, así que esos datos sí se cargan. Lo más probable es que en `cargar_datos_de_prueba` no se hayan pegado todos los `INSERT`. En `main.py` eran las líneas 24 a 53, y faltan los de `solicitudes` (y seguramente los de `historial`).

Para confirmarlo, contá los `INSERT` por tabla:

```
foreach ($t in "estudiantes","responsables","solicitudes","historial") {
  "$t : " + (Select-String -Path app\seed.py -Pattern "INSERT INTO $t ").Count
}
```

Tiene que dar:

Tabla	INSERT
estudiantes	4
responsables	5
solicitudes	6
historial	15

Si falta alguno, copialos de `main.py` en `dev` (líneas 24 a 53, que son 30 líneas en total), con la misma sangría de 4 espacios. Si ya los borraste de `main.py`, los ves así:

```
git show dev:main.py | Select-Object -Skip 23 -First 30
```

Fijate también que `conn.commit()` sea la **última** línea de `cargar_datos_de_prueba`, después de todos los `INSERT`.

Usuario (Cristian Morales) 06/11/2026 11:18

```
"""Datos de prueba sintéticos del sistema."""


def base_vacia(conn):
    """Indica si la base todavía no tiene estudiantes cargados."""
```

```
    c = conn.cursor()
    c.execute("SELECT COUNT(*) FROM estudiantes")
    return c.fetchone()[0] == 0


def cargar_datos_de_prueba(conn):
    """Carga los estudiantes, responsables, solicitudes e historial de prueba."""
    c = conn.cursor()
    # PEGAR ACÁ los c.execute("INSERT ...") de main.py (líneas 24–53),
    # con la misma sangría que ya tienen (4 espacios)
    conn.commit()


creo que me falto armar el seed, pasamelo completo
```

Claude 06/11/2026 11:18

Sí, era eso: `cargar_datos_de_prueba` no cargaba nada. Los tests de estudiantes y responsables pasaban porque crean sus propios datos. Te paso `app/seed.py` completo:

```
"""Datos de prueba sintéticos del sistema."""


def base_vacia(conn):
    """Indica si la base todavía no tiene estudiantes cargados."""
    c = conn.cursor()
    c.execute("SELECT COUNT(*) FROM estudiantes")
    return c.fetchone()[0] == 0


def cargar_datos_de_prueba(conn):
    """Carga los estudiantes, responsables, solicitudes e historial de prueba."""
    c = conn.cursor()
    c.execute("INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
('Lucia','Ferreyra','TUP-1001','lucia.ferreyra@ejemplo.com','TUP')")
    c.execute("INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
('Mateo','Quiroga','TUP-1002','mateo.quiroga@ejemplo.com','TUP')")
    c.execute("INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
('Sofia','Aranda','TUP-1003','sofia.aranda@ejemplo.com','TUP')")
    c.execute("INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
('Tomas','Villalba','TUP-1004','tomas.villalba@ejemplo.com','TUP')")
    c.execute("INSERT INTO responsables (nombre, area, email, activo) VALUES ('Graciela
Molina','Alumnado','gmolina@ejemplo.com',1)")
    c.execute("INSERT INTO responsables (nombre, area, email, activo) VALUES ('Ruben
Castro','Alumnado','rcastro@ejemplo.com',0)")
    c.execute("INSERT INTO responsables (nombre, area, email, activo) VALUES ('Paula
Diaz','Departamento Academico','pdiaz@ejemplo.com',1)")
    c.execute("INSERT INTO responsables (nombre, area, email, activo) VALUES ('Hector
Ruiz','Bedelia','hruiz@ejemplo.com',1)")
    c.execute("INSERT INTO responsables (nombre, area, email, activo) VALUES ('Marina
Sosa','Coordinacion','msosa@ejemplo.com',1)")
    c.execute("INSERT INTO solicitudes (estudiante_id, tipo, descripcion, prioridad, estado,
responsable_id, area, fecha_alta, fecha_mod, canal, obs) VALUES (1,'CERT','Necesito certificado
de alumna regular para la obra social',3,'PEND',NULL,'Alumnado','2026-09-01 10:15:00','2026-09-
01 10:15:00','WEB','')")
    c.execute("INSERT INTO solicitudes (estudiante_id, tipo, descripcion, prioridad, estado,
responsable_id, area, fecha_alta, fecha_mod, canal, obs) VALUES (2,'EQUIV','Pido equivalencia de
Programacion I aprobada en otra universidad',2,'ASIG',3,'Departamento Academico','2026-09-03
09:00:00','2026-09-04 11:20:00','MAIL','adjunta programa')")
    c.execute("INSERT INTO solicitudes (estudiante_id, tipo, descripcion, prioridad, estado,
responsable_id, area, fecha_alta, fecha_mod, canal, obs) VALUES (3,'MESA','Solicito mesa
especial de Matematica por superposicion de fechas',1,'EN_PROC',4,'Bedelia','2026-09-10
```

```
14:30:00','2026-09-12 08:45:00','WEB','')")
    c.execute("INSERT INTO solicitudes (estudiante_id, tipo, descripcion, prioridad, estado,
responsable_id, area, fecha_alta, fecha_mod, canal, obs) VALUES (4,'RECLAMO','La nota del
parcial 1 no figura cargada en el sistema',1,'RES',5,'Coordinacion','2026-09-15 16:00:00','2026-
09-16 10:00:00','PRESENCIAL','')")
    c.execute("INSERT INTO solicitudes (estudiante_id, tipo, descripcion, prioridad, estado,
responsable_id, area, fecha_alta, fecha_mod, canal, obs) VALUES (1,'BAJA','Quiero dar de baja la
materia Ingles II este cuatrimestre',2,'RECH',1,'Alumnado','2026-09-20 12:00:00','2026-09-22
09:30:00','WEB','')")
    c.execute("INSERT INTO solicitudes (estudiante_id, tipo, descripcion, prioridad, estado,
responsable_id, area, fecha_alta, fecha_mod, canal, obs) VALUES (2,'CERT','Certificado analitico
parcial urgente para presentar en trabajo',1,'PEND',NULL,'Alumnado','2026-10-01 08:10:00','2026-
10-01 08:10:00','WEB','')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (1,NULL,'PEND','2026-09-01 10:15:00','alta de solicitud')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (2,NULL,'PEND','2026-09-03 09:00:00','alta de solicitud')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (2,'PEND','ASIG','2026-09-04 11:20:00','asignada a Paula Diaz')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (3,NULL,'PEND','2026-09-10 14:30:00','alta de solicitud')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (3,'PEND','ASIG','2026-09-11 09:00:00','asignada a Hector Ruiz')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (3,'ASIG','EN_PROC','2026-09-12 08:45:00','')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (4,NULL,'PEND','2026-09-15 16:00:00','alta de solicitud')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (4,'PEND','ASIG','2026-09-15 17:00:00','asignada a Marina Sosa')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (4,'ASIG','EN_PROC','2026-09-16 08:00:00','')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (4,'EN_PROC','RES','2026-09-16 10:00:00','nota cargada')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (5,NULL,'PEND','2026-09-20 12:00:00','alta de solicitud')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (5,'PEND','ASIG','2026-09-21 10:00:00','asignada a Graciela Molina')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (5,'ASIG','EN_PROC','2026-09-21 15:00:00','')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (5,'EN_PROC','RECH','2026-09-22 09:30:00','fuera de termino')")
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (6,NULL,'PEND','2026-10-01 08:10:00','alta de solicitud')")
    conn.commit()
```

Son las 30 líneas de `main.py` (24 a 53), sin cambios: 4 estudiantes, 5 responsables, 6 solicitudes y 15 de historial.

Ojo con un detalle: si tenías el servidor levantado, se creó un `solicitudes.db` con las tablas vacías. Como `base_vacia` mira si hay estudiantes, esa base no se va a volver a cargar. Para los tests no importa, porque cada uno usa una base nueva. Pero antes de levantar el sistema, corré `.\run.bat reset`.

Después, `.\run.bat test` tendría que dar **47 passed**.

Usuario (Cristian Morales) 06/11/2026 11:19

========== warnings summary ==========
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with

`starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========== 47 passed, 1 warning in 1.00s ==========
git add main.py app/
git commit -m "refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y app/seed.py"
On branch refactor/conexion
nothing to commit, working tree clean
git status
On branch refactor/conexion
nothing to commit, working tree clean

Claude 06/11/2026 11:19

Los 47 tests pasan, así que el código está bien. Pero que diga "*nothing to commit, working tree clean*" significa que **esos cambios ya están commiteados**: o los commiteaste antes de corregir el seed, o aplicaste los patches del zip.

Antes de seguir, fijate qué hay en la rama:

```
git log --oneline dev..HEAD
git show --stat HEAD
```

Según lo que veas:

- **Aparecen los 4 commits** (`extraer la creacion...`, `reemplazar la conexion global...`, `mover los endpoints...`, `decision tecnica...`): aplicaste los patches y está todo hecho. Solo falta `git push -u origin refactor/conexion`.
- **Aparece un solo commit, el del paso 1:** quedó commiteado con el seed vacío, y la corrección del seed también se metió ahí (por ejemplo, con `--amend`). Revisá con `git show HEAD -- app/seed.py` que el commit tenga los 30 `INSERT`. Si los tiene, seguí con el paso 2.
- **No aparece ningún commit:** los cambios están en otra rama. Pasame la salida de `git branch` y `git log --oneline -5`.

Pasame lo que te muestra y seguimos.

Usuario (Cristian Morales) 06/11/2026 11:20

git show --stat HEAD
commit 19756417c0efa2e240c47856c8178d6fe71744ed (HEAD -> refactor/conexion)
Author: Cristian Ariel Morales <cristian.a.morales@gmail.com>
Date: Fri Nov 6 11:19:09 2026 -0300

    refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y app/seed.py

 app/db.py | 11 ++++++--
 app/seed.py | 92 ++++++

```
git log --oneline dev..HEAD
```

Si ves dos commits con el mismo mensaje, conviene unirlos en uno. Un commit que deja los 47 tests rotos ensucia la evidencia de "refactoring seguro", porque la consigna pide que cada refactoring tenga su commit y que los tests pasen antes y después. Como todavía no hiciste push, se puede arreglar sin riesgo:

```
git reset --soft HEAD~1
git commit --amend --no-edit
git log --oneline dev..HEAD
.\run.bat test
```

1. `reset --soft HEAD~1` deshace el último commit, pero deja sus cambios preparados.
2. `commit --amend --no-edit` los suma al commit anterior, con el mismo mensaje.
3. El log tiene que mostrar **un solo commit**, y los tests, **47 passed**.

Si en el log aparece un solo commit, o dos con mensajes distintos, no hagas nada de esto y pasame la salida.

Después seguí con el **paso 2** del paso a paso.

Usuario (Cristian Morales) 06/11/2026 11:20

========== warnings summary ==========
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========== 47 passed, 1 warning in 1.01s ==========
git log --oneline dev..HEAD
c7fb28c (HEAD -> refactor/conexion) refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y app/seed.py

Claude 06/11/2026 11:20

Quedó bien: un solo commit, con los 47 en verde. Hacé una última verificación antes de seguir:

```
git log --oneline -3
git show --stat HEAD
```

- En el `log`, la línea de abajo del commit tiene que ser `94f8215 Merge pull request #9...`, es decir, el último de `dev`.
- El `show --stat` tiene que listar **cuatro archivos**: `main.py`, `app/__init__.py`, `app/db.py` y `app/seed.py`.

Si está así, el paso 1 está cerrado y seguís con el **paso 2**: la conexión por request, del 2.1 al 2.8 del paso a paso. Es el más largo. Si algo no da 47, pasame el error.

Usuario (Cristian Morales) 06/11/2026 11:21

git log --oneline -3
c7fb28c (HEAD -> refactor/conexion) refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y app/seed.py

94f8215 (origin/dev, origin/HEAD, dev) Merge pull request #9 from CMorales2703/test/bruno-endpoints
a1fcb4b Merge pull request #12 from CMorales2703/test/javier
git show --stat HEAD
commit c7fb28c4277d3fe534edae7edc15adf64f2d7447 (HEAD -> refactor/conexion)
Author: Cristian Ariel Morales <cristian.a.morales@gmail.com>
Date: Fri Nov 6 11:16:03 2026 -0300

    refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y app/seed.py

 app/__init__.py | 0
 app/db.py | 25 +++++
 app/seed.py | 104 ++++++++++++++++++++
 main.py | 328 ++++++++++++++++++++

```
def base_vacia(conn):
    """Indica si la base todavÃa no tiene estudiantes cargados."""
    c = conn.cursor()
    c.execute("SELECT COUNT(*) FROM estudiantes")
    return c.fetchone()[0] == 0


def cargar_datos_de_prueba(conn):
    """Carga los estudiantes, responsables, solicitudes e historial de prueba."""
    c = conn.cursor()
    c.execute(
        "INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
('Lucia','Ferreyra','TUP-1001','lucia.ferreyra@ejemplo.com','TUP')"
    )
    c.execute(
        "INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
('Mateo','Quiroga','TUP-1002','mateo.quiroga@ejemplo.com','TUP')"
    )
    c.execute(
        "INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
('Sofia','Aranda','TUP-1003','sofia.aranda@ejemplo.com','TUP')"
    )
    c.execute(
        "INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
('Tomas','Villalba','TUP-1004','tomas.villalba@ejemplo.com','TUP')"
    )
```

Claude 06/11/2026 11:22

▸ *Ejecutó un comando: Preparar los archivos del paso 1*

▸ *Envió el archivo paso1.zip*

No son los finales de línea: **tu editor formatea el código al guardar.** Se ve en el seed: cada `c.execute("INSERT ...")`, que era una sola línea, quedó partido en tres. Es lo que hace un formateador como Black o Ruff cuando está activado *Format On Save*. Pasó lo mismo con `main.py`: lo reformateó entero (238 líneas agregadas, cuando deberían ser unas 8).

Los acentos rotos (`sintÃ©ticos`) no son un problema del archivo: es PowerShell, que muestra mal el UTF-8. En el editor y en GitHub se ven bien.

Cómo arreglarlo

1. Desactivá el formateo al guardar. En VS Code: `Ctrl + ,`, buscá **Format On Save** y destildalo. Si no, lo va a volver a hacer en el paso 2 con cada archivo que toques.

2. Reemplazá los tres archivos por los de `paso1.zip`. Tienen el código del paso 1 sin formatear: `main.py`, `app/db.py` y `app/seed.py`. Copialos encima de los tuyos, en las mismas rutas, y **no los abras ni los guardes** en el editor hasta haber desactivado lo del paso 1.

3. Corregí el commit:

```
.\run.bat test
git add main.py app/
git commit --amend --no-edit
git show --stat HEAD
```

Tiene que dar `47 passed`, y el `--stat` algo así:

```
 app/__init__.py |  0
 app/db.py       | 25 +++++
 app/seed.py     | 44 +++++++
 main.py         | 54 +++-----
```

`main.py` con unas 54 líneas cambiadas, no 328. Así, en el PR se va a ver claramente qué se sacó de `main.py` y adónde fue.

Después seguís con el paso 2, con el formateo ya desactivado.

Usuario (Cristian Morales) 06/11/2026 11:22

create mode 100644 app/seed.py
git show --stat HEAD
commit ddd47c24ad35b44f2811d3cee6b12f1b55ccdd4c (HEAD -> refactor/conexion)
Author: Cristian Ariel Morales <cristian.a.morales@gmail.com>
Date: Fri Nov 6 11:16:03 2026 -0300

    refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y app/seed.py

 app/__init__.py | 0
 app/db.py | 25 +++++
 app/seed.py | 104 ++++++++++++++++++++
 main.py | 328 ++++++++++++++++++++

Usuario (Cristian Morales) 06/11/2026 11:23

estan bien los cambios? porque ocn eso me sirve, sigamos con le paso 2

Claude 06/11/2026 11:24

Sí, los cambios funcionan: los 47 tests pasan, y la lógica es la misma. Lo único que se pierde es la prolijidad del diff, porque en el PR `main.py` va a aparecer reformateado entero. Si lo dejás así, anotalo en la descripción del PR ("el commit 1 incluye el formateo automático de `main.py`") para que quien revise no se confunda. Es una decisión válida.

Si seguís con el formateador activado, al menos queda todo en el mismo estilo. Lo importante es no mezclarlo: o todo formateado, o nada.

Paso 2

Seguí los puntos 2.1 a 2.8 del paso a paso. Hay dos cosas a tener en cuenta por el formateo:

- **Las firmas pueden estar partidas en varias líneas.** Black separa los parámetros largos, uno por línea. Agregá `conn: sqlite3.Connection = Depends(get_db)` como **último** parámetro, antes del `):`, esté donde esté:

`python def buscar( estado: str = None, ... orden: str = "fecha", conn: sqlite3.Connection = Depends(get_db), ): c = conn.cursor()`

- **Hay 12 lugares para tocar.** Revisá que no se te escape ninguno:

Qué	Dónde
`@app.` → `@router.` + `conn = c = conn.cursor()`	`crear_estudiante`, `listar_estudiantes`, `nuevoResponsable`, `get_resp`, `registrar`, `buscar`, `getSolicitud`, `proc_estado`, `asignar`, `reporte`
`conn` como primer parámetro + `c = conn.cursor()`	`guardar_solicitud`
pasar `conn` en la llamada	`guardar_solicitud(conn, ...)` dentro de `registrar`

Para verificar que no quedaron restos de la conexión global antes de correr los tests:

```
Select-String -Path main.py -Pattern "^conn|^c = |@app\."
```

No tiene que encontrar nada. Después, `.\run.bat test`: **47 passed**. Si falla, pasame el primer error completo, no solo la lista.

Usuario (Cristian Morales) 06/11/2026 11:25

pasame los archivos completos ya modificados

Claude 06/11/2026 11:25

▸ *Ejecutó un comando: Preparar los archivos del paso 2*

▸ *Envió el archivo paso2.zip*

Te mandé `paso2.zip` con los 4 archivos del paso 2 ya modificados, en las rutas donde van:

Archivo	Qué cambia
`main.py`	sin `conn` ni `c` globales; endpoints en `router` con `Depends(get_db)`; `preparar_base` y `crear_app` al final
`app/db.py`	agrega `get_db`
`tests/conftest.py`	`client` con `crear_app(base temporal)` y el nuevo `conexion_db`
`tests/test_reporte.py`	el test que borra el dato malo usa `conexion_db` (los asserts no cambian)

`app/seed.py` no cambia en este paso.

Copialos desde la consola, para que no los toque el formateador:

```
Expand-Archive -Path "$env:USERPROFILE\Downloads\paso2.zip" -DestinationPath "$env:TEMP\paso2" -Force
```

```
Copy-Item "$env:TEMP\paso2\main.py" .\main.py -Force
Copy-Item "$env:TEMP\paso2\app\db.py" .\app\db.py -Force
Copy-Item "$env:TEMP\paso2\tests\conftest.py" .\tests\conftest.py -Force
Copy-Item "$env:TEMP\paso2\tests\test_reporte.py" .\tests\test_reporte.py -Force
.\run.bat test
```

Tiene que dar **47 passed**. Después:

```
git add main.py app/db.py tests/conftest.py tests/test_reporte.py
git commit -m "refactor: reemplazar la conexion global por una conexion por request"
```

Un aviso: estos archivos están **sin formatear**. Como tu commit 1 quedó formateado, el diff de `main.py` entre los dos commits va a volver a mostrar muchas líneas cambiadas. Para que quede consistente, después de copiarlos podés abrir y guardar `main.py` y `app/db.py` en VS Code, así el formateador los deja con el mismo estilo que el commit 1. Corré los tests de nuevo antes de commitear.

Cuando lo tengas, seguimos con el paso 3.

Usuario (Cristian Morales) 06/11/2026 11:27

```
=================================================================
FAILURES
=================================================================
_____________________________ test_DEFECTO_el_reporte_se_recupera_al_quitar_el_dato_malo
_____________________________

client_sin_excepciones = <starlette.testclient.TestClient object at 0x000001E9F0A645A0>,
solicitud_con_fecha_invalida = None

    def test_DEFECTO_el_reporte_se_recupera_al_quitar_el_dato_malo(
            client_sin_excepciones, solicitud_con_fecha_invalida):
        """
        El fallo viene del dato y no deja el sistema en un estado inconsistente:
        con la solicitud defectuosa el reporte devuelve 500, y al borrarla vuelve
        a responder 200 sin reiniciar nada.
        """
        import sys

        assert client_sin_excepciones.get("/reporte").status_code == 500

        main = sys.modules["main"]
>       main.c.execute("DELETE FROM solicitudes WHERE descripcion LIKE 'Test - %'")
        ^^^^^^
E       AttributeError: module 'main' has no attribute 'c'

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\tests\test_reporte.py:69: AttributeError
=================================================== warnings summary ===================================================
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa
```

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========== short test summary info ==========
FAILED tests/test_reporte.py::test_DEFECTO_el_reporte_se_recupera_al_quitar_el_dato_malo - AttributeError: module 'main' has no attribute 'c'
========== 1 failed, 46 passed, 1 warning in 1.12s ==========
Presione una tecla para continuar . . .

Claude 06/11/2026 11:28

Casi: 46 de 47. El que falla todavía tiene el código viejo (`main.c.execute(...)`), así que **`tests/test_reporte.py` no se reemplazó**. Los otros tres archivos sí se copiaron, porque si no fallarían todos.

Seguramente el `Copy-Item` de ese archivo no se ejecutó, o dio un error que pasó de largo. Copialo de nuevo y verificá:

```
Copy-Item "$env:TEMP\paso2\tests\test_reporte.py" .\tests\test_reporte.py -Force
Select-String -Path tests\test_reporte.py -Pattern "conexion_db|main\.c"
```

Tiene que encontrar `conexion_db` (en los parámetros y en las dos líneas del `DELETE`) y **ninguna** línea con `main.c`.

Si preferís editarlo a mano, en ese test solo hay que cambiar esto:

```
def test_DEFECTO_el_reporte_se_recupera_al_quitar_el_dato_malo(
        client_sin_excepciones, solicitud_con_fecha_invalida, conexion_db):
    """
    El fallo viene del dato y no deja el sistema en un estado inconsistente:
    con la solicitud defectuosa el reporte devuelve 500, y al borrarla vuelve
    a responder 200 sin reiniciar nada.
    """
    assert client_sin_excepciones.get("/reporte").status_code == 500

    conexion_db.execute("DELETE FROM solicitudes WHERE descripcion LIKE 'Test - %'")
    conexion_db.commit()

    assert client_sin_excepciones.get("/reporte").status_code == 200
```

Se suma `conexion_db` a los parámetros y se saca el `import sys`. Las tres líneas de `main` pasan a ser dos con `conexion_db`. Los dos `assert` quedan iguales.

Después, `.\run.bat test`: **47 passed**, y commiteás.

Usuario (Cristian Morales) 06/11/2026 11:29

sigamos

Claude 06/11/2026 11:29

▸ *Ejecutó un comando: Preparar los archivos de los pasos 3 y 4*

▸ *Envió los archivos paso3.zip, 01_estructura_y_conexion.md*

Te mandé los archivos de los dos pasos que faltan. Antes de seguir, confirmá que el paso 2 quedó commiteado: `git log --oneline dev..HEAD` tiene que mostrar 2 commits.

Paso 3: un router por área (`paso3.zip`)

Trae `main.py` (que queda solo con `preparar_base` y `crear_app`) y la carpeta `app/routers/`, con `__init__.py` y los 5 routers. Las funciones son las mismas de `main.py`, sin cambios: solo cambian de archivo.

```
Expand-Archive -Path "$env:USERPROFILE\Downloads\paso3.zip" -DestinationPath "$env:TEMP\paso3" -
Force
Copy-Item "$env:TEMP\paso3\main.py" .\main.py -Force
Copy-Item "$env:TEMP\paso3\app\routers" .\app\ -Recurse -Force
Get-ChildItem app\routers
.\run.bat test
```

En `app\routers` tienen que aparecer 6 archivos, y los tests tienen que dar **47 passed**. Después:

```
git add main.py app/routers/
git commit -m "refactor: mover los endpoints de main.py a un router por area"
```

Paso 4: la decisión técnica (`01_estructura_y_conexion.md`)

```
New-Item -ItemType Directory -Force docs\decisiones
Copy-Item "$env:USERPROFILE\Downloads\01_estructura_y_conexion.md" docs\decisiones\ -Force
git add docs/decisiones/
git commit -m "docs: decision tecnica de estructura del proyecto y conexion a la base"
```

Verificación y push

```
git log --oneline dev..HEAD
.\run.bat test
git push -u origin refactor/conexion
```

El log tiene que mostrar **4 commits**, y los tests, **47 passed**.

Sobre el formateo: si abrís y guardás los routers en VS Code, el formateador los va a reformatear. Es aceptable, pero en ese caso hacelo **antes** del `git add` del paso 3, para que cada commit quede en un solo estilo.

Cuando abras el PR a `dev`, pasame el link y lo reviso.

Usuario (Cristian Morales) 06/11/2026 11:31

git log --oneline dev..HEAD
f1fb090 (HEAD -> refactor/conexion) refactor: reemplazar la conexion global por una conexion por request
ddd47c2 refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y app/seed.py

Claude 06/11/2026 11:31

Bien: los dos primeros pasos están commiteados. Seguí con el **paso 3** (`paso3.zip`) y después con el **paso 4** (`01_estructura_y_conexion.md`), con los comandos de mi mensaje anterior.

Cuando termines, el `git log --oneline dev..HEAD` tiene que mostrar los 4 commits:

```
... docs: decision tecnica de estructura del proyecto y conexion a la base
... refactor: mover los endpoints de main.py a un router por area
f1fb090 refactor: reemplazar la conexion global por una conexion por request
ddd47c2 refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y app/seed.py
```

Y `.\run.bat test` tiene que dar **47 passed** antes del push.

Usuario (Cristian Morales) 06/11/2026 11:34

.\run.bat test
ImportError while loading conftest
'C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\tests\conftest.py'.

tests\conftest.py:11: in <module>
    from main import crear_app
main.py:5: in <module>
    from app.routers import consultas, estados, estudiantes, responsables, solicitudes
E   ImportError: cannot import name 'consultas' from 'app.routers' (unknown location)

del paso 3

Claude 06/11/2026 11:34

El error "*(unknown location)*" indica que Python encuentra la carpeta `app\routers`, pero sin los archivos adentro. Casi seguro quedaron un nivel más abajo, en **`app\routers\routers\`**: si la carpeta `app\routers` ya existía (por ejemplo, porque la creaste siguiendo el paso a paso manual), `Copy-Item` copia la carpeta *dentro* de ella.

Fijate:

```
Get-ChildItem app\routers -Recurse
```

Si ves `app\routers\routers\...`, arreglalo así:

```
Remove-Item app\routers -Recurse -Force
New-Item -ItemType Directory app\routers | Out-Null
Copy-Item "$env:TEMP\paso3\app\routers\*" .\app\routers\ -Force
Get-ChildItem app\routers
```

Tienen que quedar **6 archivos directamente** en `app\routers`:

```
__init__.py
consultas.py
estados.py
estudiantes.py
responsables.py
solicitudes.py
```

Después, `.\run.bat test`: **47 passed**.

Si `Get-ChildItem` muestra la carpeta vacía o con otros archivos, pasame la salida.

Usuario (Cristian Morales) 06/11/2026 11:34

Get-ChildItem app\routers -Recurse

    Directorio: C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\ms2\tpi\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\app\routers

Mode   LastWriteTime   Length Name
----   -------------   ------ ----
d-----   06/11/2026 11:33 a. m.   __pycache__
-a----   06/11/2026 11:32 a. m.   3754 consultas (1).py
-a----   06/11/2026 11:32 a. m.   3268 estados (1).py
-a----   06/11/2026 11:32 a. m.   1626 estudiantes (1).py
-a----   06/11/2026 11:32 a. m.   1254 responsables.py
-a----   06/11/2026 11:32 a. m.   3489 solicitudes (1).py
-a----   06/11/2026 11:32 a. m.   0 __init__ (1).py

Directorio:

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\ms2\tpi\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\app\routers__pycache__

Mode LastWriteTime Length Name
---- ------------- ------ ----
-a---- 06/11/2026 11:33 a. m. 2644 responsables.cpython-314.pyc

Claude 06/11/2026 11:34

Ahí está: los archivos se guardaron con **`(1)` en el nombre** (`consultas (1).py`, `__init__ (1).py`...). Windows hace eso cuando descargás un archivo que ya existe en Descargas. Python no los reconoce como `consultas` ni como `__init__`, y por eso no los encuentra. Solo `responsables.py` quedó bien.

Renombralos desde la raíz del repo:

```
Get-ChildItem app\routers -Filter "* (1).py" | Rename-Item -NewName { $_.Name -replace ' \(1\)', '' }
Remove-Item app\routers\__pycache__ -Recurse -Force
Get-ChildItem app\routers
```

Tienen que quedar exactamente estos 6:

```
__init__.py
consultas.py
estados.py
estudiantes.py
responsables.py
solicitudes.py
```

Después, `.\run.bat test`: **47 passed**. Y seguí con el commit del paso 3.

Revisá también tu carpeta Descargas: si ahí hay archivos con `(1)` de los zips anteriores, borralos antes de descargar otro, para que no se mezclen.

Usuario (Cristian Morales) 06/11/2026 11:35

Copy-Item "$env:USERPROFILE\Downloads\01_estructura_y_conexion.md" docs\decisiones\ -Force
Copy-Item : No se encuentra la ruta de acceso
'C:\Users\crist\Downloads\01_estructura_y_conexion.md' porque no existe.
En línea: 1 Carácter: 1
+ Copy-Item "$env:USERPROFILE\Downloads\01_estructura_y_conexion.md" do ...
+
~~~
~~~

deleted: app/__init__.py

no changes added to commit (use "git add" and/or "git commit -a")

Claude 06/11/2026 11:35

Hay dos cosas, y una es importante.

1. Se borró `app/__init__.py`

El `git status` muestra `deleted: app/__init__.py`. Seguramente se borró al limpiar los `(1)`. Sin ese archivo, `app` deja de ser un paquete. Recuperalo desde el último commit:

```
git restore app/__init__.py
```

2. El `.md` no está en Descargas con ese nombre

Probablemente quedó como `01_estructura_y_conexion (1).md`, o en otra carpeta. Buscalo:

```
Get-ChildItem "$env:USERPROFILE\Downloads" -Filter "01_estructura*"
```

Copialo con el nombre correcto, ajustando el origen al que aparezca:

```
New-Item -ItemType Directory -Force docs\decisiones | Out-Null
Copy-Item "$env:USERPROFILE\Downloads\01_estructura_y_conexion (1).md"
docs\decisiones\01_estructura_y_conexion.md -Force
```

Si no aparece en Descargas, volvé a descargarlo del archivo que te mandé.

Antes del commit del paso 4

Confirmá que el paso 3 quedó commiteado y que todo está en orden:

```
git log --oneline dev..HEAD
git status
.\run.bat test
```

- El log tiene que mostrar **3 commits**, el último `mover los endpoints...`. Si muestra 2, falta commitear el paso 3: `git add main.py app/routers/` y el commit.
- El `status` solo tiene que mostrar `docs/decisiones/` como nuevo, nada borrado.
- Los tests tienen que dar **47 passed**.

Recién ahí:

```
git add docs/decisiones/
git commit -m "docs: decision tecnica de estructura del proyecto y conexion a la base"
```

Usuario (Cristian Morales) 06/11/2026 11:37

bien todo commiteado, verificamos

Claude 06/11/2026 11:37

▸ *Ejecutó un comando: Verificar si la rama está en GitHub*

Todavía no la puedo ver: la rama `refactor/conexion` no está en GitHub, así que falta el push. Yo solo puedo revisar lo que está subido.

Antes de subirla, hacé estas verificaciones locales:

```
git log --oneline dev..HEAD
git status
```

```
git show --stat HEAD~1
.\run.bat test
```

Lo que tenés que ver:

- **El log**, 4 commits: `extraer la creacion...`, `reemplazar la conexion global...`, `mover los endpoints...` y `decision tecnica...`.
- **El status**, `nothing to commit, working tree clean`.
- **El `show --stat HEAD~1`** (el commit del paso 3): los 6 archivos de `app/routers/` con nombres limpios, **sin `(1)`** y sin `__pycache__`.
- **Los tests**, **47 passed**.

Si todo da bien:

```
git push -u origin refactor/conexion
```

Avisame cuando esté subida y la reviso completa desde acá: los commits, los archivos, los tests, que `import main` no cree la base y que no se haya colado nada.

Usuario (Cristian Morales) 06/11/2026 11:42

git commit -m "docs: decision tecnica de estructura del proyecto y conexion a la base"
[refactor/conexion 2fb3822] docs: decision tecnica de estructura del proyecto y conexion a la base
 1 file changed, 67 insertions(+)
 create mode 100644 docs/decisiones/01_estructura_y_conexion.md
git log --oneline dev..HEAD
2fb3822 (HEAD -> refactor/conexion) docs: decision tecnica de estructura del proyecto y conexion a la base
f7730a3 refactor: mover los endpoints de main.py a un router por area
f1fb090 refactor: reemplazar la conexion global por una conexion por request
ddd47c2 refactor: extraer la creacion de tablas y los datos de prueba a app/db.py y app/seed.py
git status
On branch refactor/conexion
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted: app/__init__.py

no changes added to commit (use "git add" and/or "git commit -a")
git show --stat HEAD~1
commit f7730a3774df67313b02b26cf48e46e02401ed21
Author: Cristian Ariel Morales <cristian.a.morales@gmail.com>
Date: Fri Nov 6 11:35:29 2026 -0300

    refactor: mover los endpoints de main.py a un router por area

 app/routers/__init__.py | 0
 app/routers/consultas.py | 105 +++++++++++
 app/routers/estados.py | 82 +++++++
 app/routers/estudiantes.py | 41 ++++
 app/routers/responsables.py | 34 +++
 app/routers/solicitudes.py | 97 ++++++++
 main.py | 537 +--
 7 files changed, 370 insertions(+), 526 deletions(-)
.\run.bat test
=== test session starts ===

platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0
rootdir:
C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 47 items

tests\test_asignacion.py .... [8%]
tests\test_buscar.py ..... [34%]
tests\test_detalle.py ..... [46%]
tests\test_estado.py ..... [57%]
tests\test_estudiantes.py ..... [78%]
tests\test_reporte.py ..... [93%]
tests\test_responsables.py ... [100%]

========== warnings summary ==========
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========== 47 passed, 1 warning in 1.01s ==

Claude 06/11/2026 11:42

Los 4 commits están, y el commit del paso 3 tiene los 6 routers con nombres limpios. Los tests dan **47 passed**.

Solo queda una cosa: `app/__init__.py` **sigue borrado** en tu carpeta. No lo restauraste. Los tests pasan igual, porque Python moderno acepta carpetas sin `__init__.py`, pero el archivo está en el commit 1 y conviene que siga ahí. Si no, alguien hace `git add -A` más adelante y lo borra del repo sin querer. Restauralo:

```
git restore app/__init__.py
git status
```

El `status` tiene que decir `nothing to commit, working tree clean`. Como el archivo ya está commiteado, no hace falta un commit nuevo.

Después, subí la rama:

```
git push -u origin refactor/conexion
```

Avisame y la reviso desde acá.

Usuario (Cristian Morales) 06/11/2026 13:48

https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta

ahora si estamos para hacer el pr a main

Claude 06/11/2026 13:49

▸ *Ejecutó un comando: Revisar la rama refactor/conexion, dev, main y PRs*

▸ *Ejecutó un comando: Combinar la rama con dev y correr todos los tests*

▸ *Ejecutó un comando: Leer la tabla de tags del README*

Revisé la rama y está bien. Pero el PR no va a `main` todavía: va a `dev`.

La revisión

- **4 commits limpios** sobre `dev`. Sin `(1)`, sin `__pycache__`, y con `app/__init__.py` presente.
- **Combinada con el `dev` actual da 64 passed.** Ya están mergeados los tests de Gaston (PR #14), y pasan con la nueva estructura sin cambios. Esa era la condición que faltaba.
- **No hay conflictos** con `dev`.
- **`import main` en una carpeta vacía no crea nada.** Tus hallazgos 1 y 2 quedaron resueltos.

Por qué no a `main`

Según lo que acordaron (y la tabla de tags del README), a `main` se pasa al **cerrar cada etapa**. La etapa de refactoring cierra cuando los cuatro terminen los suyos, y ahí se pone `etapa-refactoring`. Tus transversales son el **primer paso** de esa etapa, no el cierre. Si los pasás ahora a `main`, quedaría marcada una etapa a medias.

Ya existe el **PR #13** con tu rama. Fijate que apunte a **`dev`** (arriba, donde dice *base*). Si apunta a `main`, cambialo con **Edit** al lado del título.

Para la descripción del PR, poné al final: "*Combinado con el `dev` actual (incluye los tests de Gaston): 64 passed. Ningún assert modificado.*" Y como el commit 1 quedó formateado, aclará: "*El commit 1 incluye el formateo automático de `main.py`.*"

Pedile la revisión a Javier y mergealo con **"Create a merge commit"**.

Una decisión para el grupo

`dev` tiene los tests de la Fase 2, que todavía no están en `main`. Si quieren que `main` muestre la línea de base **antes** del refactoring, hagan ahora un PR `dev` → `main`, **antes de mergear el #13**. Según la tabla del README, no lleva tag, porque los tests de línea de base no son una etapa con tag propio. Es opcional, pero suma claridad a la historia de `main`.

Después del merge del #13, avisá al grupo que ya pueden arrancar cada uno con su router.

Usuario (Cristian Morales) 06/11/2026 13:50

si, decime como seguir para que lso cambios de test queden bajo el tag de estapa-test

Claude 06/11/2026 13:51

Antes, un detalle: **`etapa-test` no existe en la tabla de tags que acordaron.** La más parecida es `etapa-pruebas`, pero según el README significa "*Cierre de la verificación*", es decir, las pruebas **después** del refactoring. Si la usás ahora, el tag de esa etapa quedaría antes del refactor y no tendría con qué marcar el cierre de la verificación.

Lo prolijo es **agregar un tag nuevo** para la línea de base y dejarlo escrito en el README. Te propongo `etapa-tests`, que es el nombre que usaste. Consultalo en el grupo, porque cambia la tabla que acordaron.

Orden de los pasos

No mergees el PR #13 hasta terminar el paso 3. Si entra antes, el refactor queda dentro de la etapa de tests.

1. Actualizar el README (rama → `dev`)

```
git checkout dev
git pull
```

```
git checkout -b docs/tags-etapas
```

En `README.md`, en la tabla de tags, agregá la fila nueva debajo de `etapa-diagnostico`:

```
| `estado-inicial` | Sistema base sin modificar |
| `etapa-diagnostico` | Cierre del diagnóstico |
| `etapa-tests` | Tests de línea de base, antes del refactoring |
| `etapa-refactoring` | Cierre de los refactorings |
| `etapa-pruebas` | Cierre de la verificación |
| `etapa-docs` | Cierre de la documentación |
| `estadorefactorizado` | Versión final |
```

Y reemplazá la línea de abajo ("*Los tags de etapa se crean sobre `dev`...*") por lo que realmente hacen:

```
Al cerrar cada etapa se integra `dev` en `main` con un PR (merge commit) y el tag de la etapa se
crea sobre `main`. El tag `estadorefactorizado` marca la versión final.
```

```
git add README.md
git commit -m "docs: agregar el tag etapa-tests y aclarar que los tags de etapa van sobre main"
git push -u origin docs/tags-etapas
```

Abrí el PR a **`dev`**, que lo apruebe alguien y mergealo.

2. PR `dev` → `main`

En GitHub: **New pull request**, con base `main` y compare `dev`.

Título: `Fase 2: tests de línea de base`

Descripción:

```
## Etapa: tests de línea de base (Fase 2)

Integra en `main` los tests que fijan el comportamiento de `estado-inicial` antes del
refactoring.
Al mergear se marca con el tag `etapa-tests`.

| Integrante | Tests | PR |
|---|---|---|
| Cristian Morales | estudiantes y responsables | #4 |
| Javier Ovelar | cambio de estado y asignación | #12 |
| Bruno Maza | búsqueda, detalle y reporte | #9 |
| Gaston Paschetta | registro de solicitudes | #14 |

- Criterio: cada test verifica el comportamiento actual; el esperado queda en un comentario.
- `main.py` no cambia.
- Resultado: 64 passed.
```

Esperá la aprobación y mergealo con **"Create a merge commit"**.

3. Tag sobre `main`

```
git checkout main
git pull
git log --oneline -1
git tag -a etapa-tests -m "Fase 2: tests de linea de base"
git push origin etapa-tests
```

El `log` tiene que mostrar el merge del PR que acabás de cerrar.

4. Recién ahora, el PR #13 a `dev`

Antes de mergearlo, en el PR #13 tocá **"Update branch"** para traer el cambio del README. Fijate que siga en verde y mergealo con **"Create a merge commit"**. Después avisá al grupo que ya

pueden arrancar con sus routers.

Cuando esté el tag, pasame el aviso y verifico desde acá que quedó sobre el commit correcto.

Usuario (Cristian Morales) 06/11/2026 13:53

git add README.md
git commit -m "docs: agregar el tag etapa-tests y aclarar que los tags de etapa van sobre main"
On branch docs/tags-etapas
nothing to commit, working tree clean
git push -u origin docs/tags-etapas
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'docs/tags-etapas' on GitHub by visiting:
remote:
https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta/pull/new/docs/tags-etapas
remote:
To https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta.git
 * [new branch]   docs/tags-etapas -> docs/tags-etapas
branch 'docs/tags-etapas' set up to track 'origin/docs/tags-etapas'.

Claude 06/11/2026 13:54

El push subió la rama, pero **vacía**: el commit dijo "*nothing to commit*", así que el README no tiene cambios. Lo más probable es que lo hayas editado sin guardarlo (`Ctrl + S`) o que hayas editado otro archivo.

Todavía no abras el PR. Hacé esto:

1. Abrí `README.md` en la raíz del repo, aplicá los dos cambios (la fila de `etapa-tests` y la línea de abajo de la tabla) y **guardá**.
2. Fijate que Git vea el cambio:

`powershell git diff README.md`

Tienen que aparecer la fila nueva con `+` y la línea de "*Los tags de etapa...*" reemplazada (una línea con `-` y otra con `+`). Si no aparece nada, el archivo no se guardó.

3. Commiteá y subí:

`powershell git add README.md git commit -m "docs: agregar el tag etapa-tests y aclarar que los tags de etapa van sobre main" git push`

Como la rama ya está en GitHub, alcanza con `git push`, sin `-u`. Después abrí el PR a `dev`.

Usuario (Cristian Morales) 06/11/2026 13:57

listo mergado a dev el cambio del readme, que sigue?

Claude 06/11/2026 13:57

▸ *Ejecutó un comando: Verificar el README en dev*

Confirmado: el README ya está en `dev` (PR #15), con la fila de `etapa-tests` y la línea nueva sobre los tags. El refactor todavía **no** entró en `dev`, que es lo correcto.

Sigue el **paso 2**: el PR de `dev` a `main`.

1. En GitHub: **Pull requests → New pull request**, con base **`main`** y compare **`dev`**.
2. **Título:** `Fase 2: tests de línea de base`
3. **Descripción:** la del mensaje anterior. Agregale una línea para el README:

`markdown - README: se agrega el tag `etapa-tests` y se aclara que los tags de etapa van sobre `main` (#15).`

4. Antes de crearlo, revisá en la pestaña **Files changed** que **no aparezcan** `app/` ni `docs/decisiones/`. Si aparecen, el refactor se coló y no hay que seguir.
5. Pedí la aprobación y mergealo con **"Create a merge commit"**.

Después, el tag sobre `main`:

```
git checkout main
git pull
git log --oneline -1
git tag -a etapa-tests -m "Fase 2: tests de linea de base"
git push origin etapa-tests
```

Y recién ahí, el PR #13 a `dev`: primero **"Update branch"** y después el merge.

Avisame cuando esté el tag y verifico que haya quedado en el commit correcto.

Usuario (Cristian Morales) 06/11/2026 14:03

listo ahi quedo creo todo

Claude 06/11/2026 14:03

▸ *Ejecutó un comando: Verificar el tag etapa-tests*

▸ *Ejecutó un comando: Correr los tests en etapa-tests y en dev*

Lo verifiqué y quedó todo bien.

	Estado
Tag `etapa-tests`	Está sobre `main`, en el merge del PR #16. Tiene los 9 archivos de tests y **no** incluye el refactor. Corriendo los tests en ese tag: **64 passed**.
`main`	Tiene el diagnóstico y los tests, sin el refactor, que es lo que corresponde.
`dev`	Ya tiene tu PR #13 con los 4 commits conservados (merge commit). Con la nueva estructura: **64 passed**.
Tags	`estado-inicial`, `etapa-diagnostico`, `etapa-tests`.

La etapa de tests quedó cerrada y tus transversales están integrados.

Lo que sigue

1. Avisá al grupo que ya pueden arrancar, cada uno con su router y en una rama `refactor/<tema>` desde `dev`:

Integrante	Archivo	Refactoring que garantiza
Gaston	`app/routers/solicitudes.py`	Introduce Parameter Object
Javier	`app/routers/estados.py`	Move Method
Bruno	`app/routers/consultas.py`	Extract Method
Cristian	`app/routers/estudiantes.py` y `responsables.py`	tu refactor grueso

Pasales también la regla: **los 64 tests tienen que seguir en verde sin cambiar ningún assert**. Cambiar códigos de error (200 a 422) es corrección de defecto, va aparte y necesita el criterio unificado del grupo.

2. Actualizá tu `docs/uso-ia/cristian.md` con una fila de la Fase 3: los transversales los escribió la IA, y vos los aplicaste, verificaste y corregiste (el seed, los `(1)`, el formateo).

3. Tu refactor grueso (estudiantes y responsables). Cuando quieras, arrancamos: primero repasamos qué hallazgos de tu ficha quedan abiertos para tus dos routers y elegís por cuál empezar.

Usuario (Cristian Morales) 06/11/2026 14:08

agreguemos la actualizacion de .md con el refactor, avancemos

Claude 06/11/2026 14:10

▸ *Ejecutó un comando: Revisar los hallazgos abiertos y los routers*
▸ *Ejecutó un comando: Renombrar los endpoints de responsables (copia local)*
▸ *Ejecutó un comando: Extraer la conversión de filas (copia local)*
▸ *Ejecutó un comando: Extraer la validación de email y las constantes (copia local)*
▸ *Ejecutó un comando: Dividir crear_estudiante (copia local)*
▸ *Ejecutó un comando: Registrar el paso 5 (copia local)*
▸ *Ejecutó un comando: Redactar el documento de la Fase 3 y la fila de uso de IA*
▸ *Ejecutó un comando: Exportar y verificar los patches del refactor*
▸ *Envió el archivo refactor-personas.zip*

Armé tu refactor de estudiantes y responsables, con la documentación y la actualización de `cristian.md` incluidas. Son 6 commits sobre el `dev` actual. Los **64 tests pasan después de cada uno, sin cambiar ningún assert**.

Los commits

#	Refactoring	Hallazgos
1	**Rename Method:** `nuevoResponsable` → `crear_responsable` y `get_resp` → `listar_responsables`	12
2	**Extract Method:** `fila_a_estudiante()` y `fila_a_responsable()`, con columnas por nombre (`fila["legajo"]` en vez de `r[3]`)	8, 9
3	**Extract Function:** la validación de email duplicada pasa a `app/validaciones.py`	4
4	**Constantes:** `AREAS` y `CARRERA_POR_DEFECTO` en `app/dominio.py`	6, 13
5	**Extract Method:** `crear_estudiante` delega en `validar_estudiante()`, `legajo_existe()` e `insertar_estudiante()`	7
6	**Docs:** `docs/refactoring/cristian_refactoring.md` con antes y después de cada refactoring, y la fila de la Fase 3 en `docs/uso-ia/cristian.md`	—

Quedan afuera, a propósito, los hallazgos **3, 5 y 11**: cambian lo que ve el cliente (de 200 a 422, rechazar nulos), así que son corrección de defecto. El documento explica por qué y quedan para cuando el grupo unifique el criterio de códigos.

Dos detalles para la defensa:

- **`row_factory` se aplica solo al cursor de tus endpoints.** Así no afecta los routers de los demás.
- **El commit 3 mantiene a propósito el 500 con `email: null`** (caso 10). Corregirlo sería un cambio de comportamiento.

Cómo aplicarlo

Igual que la vez anterior con `git am`, que no pasa por el editor ni por Descargas:

```
git checkout dev
git pull
git checkout -b refactor/personas
Expand-Archive -Path "$env:USERPROFILE\Downloads\refactor-personas.zip" -DestinationPath
"$env:TEMP\refactor-personas" -Force
git am "$env:TEMP\refactor-personas\*.patch"
git log --oneline dev..HEAD
.\run.bat test
git push -u origin refactor/personas
```

El log tiene que mostrar los 6 commits, y los tests, **64 passed**. Si `git am` falla, no sigas: corré `git am --abort` y pasame el error.

Después, abrí el PR a **`dev`**, pedile la revisión a alguien y mergealo con **merge commit**.

Pendiente

La fila de uso de IA apunta a `anexo/ia/cristian_claude_04_refactoring.pdf`. Cuando cierres la Fase 3, exportá la conversación como hicimos la otra vez y te armo el PDF.

Usuario (Cristian Morales) 06/11/2026 20:09

mejor pasame paso a paso los cambios en los archivos, asi voy haciendo los commits

Claude 06/11/2026 20:09

▸ *Ejecutó un comando: Verificar si dev cambió*

▸ *Ejecutó un comando: Preparar los documentos de la Fase 3*

▸ *Envió los archivos cristian_refactoring.md, cristian.md*

Son 6 pasos, uno por commit. Después de cada uno, corré `.\run.bat test` y confirmá que da **64 passed** antes de commitear. Si falla, no sigas: pasame el error.

Antes de arrancar: confirmá que *Format On Save* sigue desactivado en VS Code, para que el diff de cada commit muestre solo lo que cambiaste.

```
git checkout dev
git pull
git checkout -b refactor/personas
```

Paso 1: renombrar los endpoints de responsables (hallazgo 12)

En `app/routers/responsables.py`, cambiá solo los dos nombres de función:

```
# antes
def nuevoResponsable(data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)):
# después
def crear_responsable(data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)):
```

```
# antes
def get_resp(conn: sqlite3.Connection = Depends(get_db)):
# después
def listar_responsables(conn: sqlite3.Connection = Depends(get_db)):
```

```
.\run.bat test
git add app/routers/responsables.py
git commit -m "refactor: renombrar los endpoints de responsables con el criterio de estudiantes"
```

Paso 2: conversión de filas con columnas por nombre (hallazgos 8 y 9)

En `app/routers/estudiantes.py`:

1. Debajo de `router = APIRouter()`, agregá:

```python
def fila_a_estudiante(fila): """Convierte una fila de la tabla estudiantes en el diccionario de respuesta.""" return {"id": fila["id"], "nombre": fila["nombre"], "apellido": fila["apellido"], "legajo": fila["legajo"], "email": fila["email"], "carrera": fila["carrera"]} ```

2. Reemplazá el cuerpo de `listar_estudiantes`:

```python # antes c = conn.cursor() c.execute("SELECT * FROM estudiantes") rows = c.fetchall() res = [] for r in rows: res.append({"id": r[0], "nombre": r[1], "apellido": r[2], "legajo": r[3], "email": r[4], "carrera": r[5]}) return res

# después c = conn.cursor() c.row_factory = sqlite3.Row c.execute("SELECT * FROM estudiantes") return [fila_a_estudiante(fila) for fila in c.fetchall()] ```

**En `app/routers/responsables.py`:**

1. Debajo de `router = APIRouter()`:

```python
```

def fila_a_responsable(fila): """Convierte una fila de la tabla responsables en el diccionario de respuesta.""" return {"id": fila["id"], "nombre": fila["nombre"], "area": fila["area"], "email": fila["email"], "activo": fila["activo"] == 1} ```

2. Reemplazá el cuerpo de `listar_responsables`:

```python # antes c = conn.cursor() c.execute("SELECT * FROM responsables") res = [] for r in c.fetchall(): res.append({"id": r[0], "nombre": r[1], "area": r[2], "email": r[3], "activo": r[4] == 1}) return res

# después c = conn.cursor() c.row_factory = sqlite3.Row c.execute("SELECT * FROM responsables") return [fila_a_responsable(fila) for fila in c.fetchall()] ```

```
.\run.bat test
git add app/routers/estudiantes.py app/routers/responsables.py
git commit -m "refactor: extraer la conversion de filas y acceder a las columnas por nombre"
```

---

## Paso 3: validación de email en un solo lugar (hallazgo 4)

1. Creá `app/validaciones.py`:

```python """Validaciones de datos de entrada compartidas entre endpoints."""

def email_invalido(data): """Indica si el body trae un email sin '@'.

```
    Mantiene el comportamiento original: si no viene email, es válido; si viene
    en null, falla con TypeError (defecto documentado, caso 10 de la Fase 1).
    """
    return "email" in data and "@" not in data["email"]
```

```

2. En **`estudiantes.py`** y **`responsables.py`**: - debajo de `from app.db import get_db`, agregá:

   `python from app.validaciones import email_invalido`

- reemplazá la condición del email:

  `python # antes if "email" in data and "@" not in data["email"]: # después if email_invalido(data):`

  La línea de abajo (`return {"error": "email invalido"}`) no cambia.

```
.\run.bat test
git add app/validaciones.py app/routers/estudiantes.py app/routers/responsables.py
git commit -m "refactor: extraer la validacion de email duplicada a app/validaciones.py"
```

---

## Paso 4: áreas y carrera por defecto como constantes (hallazgos 6 y 13)

1. Creá `app/dominio.py`:

```python """Valores del dominio compartidos por todo el sistema."""

AREAS = ("Alumnado", "Departamento Academico", "Bedelia", "Coordinacion")

CARRERA_POR_DEFECTO = "TUP" ```

2. En **`responsables.py`**: - debajo de `from app.db import get_db`:

   `python from app.dominio import AREAS`

- en `crear_responsable`:

  `python # antes if "area" not in data or data["area"] not in ["Alumnado", "Departamento Academico", "Bedelia", "Coordinacion"]: # después if "area" not in data or data["area"] not in AREAS:`

3. En **`estudiantes.py`**: - debajo de `from app.db import get_db`:

   `python from app.dominio import CARRERA_POR_DEFECTO`

- reemplazá las **dos** apariciones de `data.get("carrera", "TUP")` por `data.get("carrera", CARRERA_POR_DEFECTO)`: una en el `INSERT` y otra en el `return`.

```
.\run.bat test
git add app/dominio.py app/routers/estudiantes.py app/routers/responsables.py
git commit -m "refactor: centralizar las areas y la carrera por defecto en app/dominio.py"
```

Paso 5: dividir `crear_estudiante` (hallazgo 7)

En `estudiantes.py`, reemplazá **toda** la función `crear_estudiante`, desde `@router.post("/estudiantes")` hasta su `return`, por:

```
def validar_estudiante(data):
    """Devuelve el mensaje de error del primer dato inválido, o None si están bien."""
    if "nombre" not in data or data["nombre"] == "" or data["nombre"] is None:
        return "falta nombre"
    if "apellido" not in data or data["apellido"] == "":
        return "falta apellido"
    if "legajo" not in data or data["legajo"] == "":
        return "falta legajo"
    if email_invalido(data):
        return "email invalido"
    return None


def legajo_existe(conn, legajo):
    """Indica si ya hay un estudiante con ese legajo."""
    c = conn.cursor()
    c.execute("SELECT * FROM estudiantes WHERE legajo = ?", (legajo,))
    return c.fetchone() is not None


def insertar_estudiante(conn, data):
    """Guarda el estudiante y devuelve el registro creado."""
    estudiante = {"nombre": data["nombre"], "apellido": data["apellido"], "legajo":
data["legajo"],
                  "email": data.get("email", ""), "carrera": data.get("carrera",
CARRERA_POR_DEFECTO)}
    c = conn.cursor()
    c.execute("INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
(?,?,?,?,?)",
              (estudiante["nombre"], estudiante["apellido"], estudiante["legajo"],
               estudiante["email"], estudiante["carrera"]))
    conn.commit()
    return {"id": c.lastrowid, **estudiante}


@router.post("/estudiantes")
def crear_estudiante(data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)):
    error = validar_estudiante(data)
    if error:
        return {"error": error}
    if legajo_existe(conn, data["legajo"]):
        return {"error": "legajo repetido"}
    return insertar_estudiante(conn, data)
```

Las validaciones son las mismas, en el mismo orden y con los mismos mensajes. Solo cambian de lugar.

```
.\run.bat test
git add app/routers/estudiantes.py
```

```
git commit -m "refactor: dividir crear_estudiante en validacion, busqueda de legajo e insercion"
```

Paso 6: documentación

Te mandé los dos archivos:

- `cristian_refactoring.md` va en **`docs/refactoring/`** (carpeta nueva): tiene el antes y después de cada refactoring;
- `cristian.md` reemplaza **`docs/uso-ia/cristian.md`**: agrega la fila de la Fase 3.

Copialos desde la consola, revisando el nombre por si Windows le agregó `(1)`:

```
New-Item -ItemType Directory -Force docs\refactoring | Out-Null
Copy-Item "$env:USERPROFILE\Downloads\cristian_refactoring.md" docs\refactoring\ -Force
Copy-Item "$env:USERPROFILE\Downloads\cristian.md" docs\uso-ia\cristian.md -Force
git status
```

El `status` tiene que mostrar solo `docs/refactoring/` como nuevo y `docs/uso-ia/cristian.md` como modificado.

```
git add docs/refactoring/ docs/uso-ia/cristian.md
git commit -m "docs: refactorings de la fase 3 (antes y despues) y registro de uso de IA"
```

Para cerrar

```
git log --oneline dev..HEAD
git status
.\run.bat test
git push -u origin refactor/personas
```

Tienen que aparecer **6 commits**, el árbol tiene que estar limpio y los tests tienen que dar **64 passed**. Después abrí el PR a `dev` y pasame el link para revisarlo.

Usuario (Cristian Morales) 06/11/2026 20:19

```
"""Estudiantes: alta y listado."""

import sqlite3

from fastapi import APIRouter, Body, Depends

from app.db import get_db
from app.dominio import CARRERA_POR_DEFECTO
from app.validaciones import email_invalido

router = APIRouter()


def fila_a_estudiante(fila):
    """Convierte una fila de la tabla estudiantes en el diccionario de respuesta."""
    return {
        "id": fila["id"],
        "nombre": fila["nombre"],
        "apellido": fila["apellido"],
        "legajo": fila["legajo"],
        "email": fila["email"],
        "carrera": fila["carrera"],
    }
```

```
@router.post("/estudiantes")
def crear_estudiante(
    data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)
):
    c = conn.cursor()
    if "nombre" not in data or data["nombre"] == "" or data["nombre"] is None:
        return {"error": "falta nombre"}
    if "apellido" not in data or data["apellido"] == "":
        return {"error": "falta apellido"}
    if "legajo" not in data or data["legajo"] == "":
        return {"error": "falta legajo"}
    if email_invalido(data):
        return {"error": "email invalido"}
    c.execute("SELECT * FROM estudiantes WHERE legajo = ?", (data["legajo"],))
    if c.fetchone():
        return {"error": "legajo repetido"}
    c.execute(
        "INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
(?,?,?,?,?)",
        (
            data["nombre"],
            data["apellido"],
            data["legajo"],
            data.get("email", ""),
            data.get("carrera", CARRERA_POR_DEFECTO),
        ),
    )
    conn.commit()
    return {
        "id": c.lastrowid,
        "nombre": data["nombre"],
        "apellido": data["apellido"],
        "legajo": data["legajo"],
        "email": data.get("email", ""),
        "carrera": data.get("carrera", CARRERA_POR_DEFECTO),
    }


def validar_estudiante(data):
    """Devuelve el mensaje de error del primer dato inválido, o None si están bien."""
    if "nombre" not in data or data["nombre"] == "" or data["nombre"] is None:
        return "falta nombre"
    if "apellido" not in data or data["apellido"] == "":
        return "falta apellido"
    if "legajo" not in data or data["legajo"] == "":
        return "falta legajo"
    if email_invalido(data):
        return "email invalido"
    return None


def legajo_existe(conn, legajo):
    """Indica si ya hay un estudiante con ese legajo."""
    c = conn.cursor()
    c.execute("SELECT * FROM estudiantes WHERE legajo = ?", (legajo,))
    return c.fetchone() is not None


def insertar_estudiante(conn, data):
    """Guarda el estudiante y devuelve el registro creado."""
    estudiante = {
        "nombre": data["nombre"],
        "apellido": data["apellido"],
```

```
        "legajo": data["legajo"],
        "email": data.get("email", ""),
        "carrera": data.get("carrera", CARRERA_POR_DEFECTO),
    }
    c = conn.cursor()
    c.execute(
        "INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
(?,?,?,?,?)",
        (
            estudiante["nombre"],
            estudiante["apellido"],
            estudiante["legajo"],
            estudiante["email"],
            estudiante["carrera"],
        ),
    )
    conn.commit()
    return {"id": c.lastrowid, **estudiante}


@router.post("/estudiantes")
def crear_estudiante(
    data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)
):
    error = validar_estudiante(data)
    if error:
        return {"error": error}
    if legajo_existe(conn, data["legajo"]):
        return {"error": "legajo repetido"}
    return insertar_estudiante(conn, data)
 asi queda el paso 5?
```

Claude 06/11/2026 20:19

Casi. Hay dos problemas, y los dos rompen cosas:

1. Quedó la versión vieja de `crear_estudiante`

La función original sigue ahí (la primera `@router.post("/estudiantes")`, con todas las validaciones adentro) y abajo agregaste la nueva. Ahora hay **dos endpoints `POST /estudiantes`**. FastAPI usa el primero que se registró, que es el viejo, así que las funciones nuevas nunca se ejecutan. Los tests pasarían igual, pero el refactor no estaría aplicado, y en el PR se vería código duplicado.

Borrá el bloque viejo completo: desde el primer `@router.post("/estudiantes")` hasta el `return { "id": c.lastrowid, ... }` que tiene `CARRERA_POR_DEFECTO` dos veces. Todo lo que está entre `fila_a_estudiante` y `def validar_estudiante`.

2. Falta `listar_estudiantes`

El archivo que me pasaste termina en `crear_estudiante`: el `GET /estudiantes` desapareció. Seguramente lo borraste al reemplazar la función. Sin él, fallan todos los tests que listan estudiantes. Agregalo al final:

```
@router.get("/estudiantes")
def listar_estudiantes(conn: sqlite3.Connection = Depends(get_db)):
    c = conn.cursor()
    c.row_factory = sqlite3.Row
    c.execute("SELECT * FROM estudiantes")
    return [fila_a_estudiante(fila) for fila in c.fetchall()]
```

Cómo tiene que quedar el orden

1. imports y `router = APIRouter()`

2. `fila_a_estudiante`
3. `validar_estudiante`
4. `legajo_existe`
5. `insertar_estudiante`
6. `crear_estudiante` (**una sola vez**, la corta)
7. `listar_estudiantes`

Para verificar:

```
Select-String -Path app\routers\estudiantes.py -Pattern "^def |^@router"
.\run.bat test
```

Tiene que listar **una** sola vez cada función, con `@router.post` y `@router.get` una vez cada uno, y los tests tienen que dar **64 passed**.

Sobre el formato: está formateado (Black), pero si los pasos anteriores también quedaron así, es consistente y está bien.

Usuario (Cristian Morales) 06/11/2026 20:19

```
"""Estudiantes: alta y listado."""

import sqlite3

from fastapi import APIRouter, Body, Depends

from app.db import get_db
from app.dominio import CARRERA_POR_DEFECTO
from app.validaciones import email_invalido

router = APIRouter()


def fila_a_estudiante(fila):
    """Convierte una fila de la tabla estudiantes en el diccionario de respuesta."""
    return {
        "id": fila["id"],
        "nombre": fila["nombre"],
        "apellido": fila["apellido"],
        "legajo": fila["legajo"],
        "email": fila["email"],
        "carrera": fila["carrera"],
    }


@router.post("/estudiantes")
def crear_estudiante(
    data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)
):
    c = conn.cursor()
    if "nombre" not in data or data["nombre"] == "" or data["nombre"] is None:
        return {"error": "falta nombre"}
    if "apellido" not in data or data["apellido"] == "":
        return {"error": "falta apellido"}
    if "legajo" not in data or data["legajo"] == "":
        return {"error": "falta legajo"}
    if email_invalido(data):
        return {"error": "email invalido"}
    c.execute("SELECT * FROM estudiantes WHERE legajo = ?", (data["legajo"],))
    if c.fetchone():
        return {"error": "legajo repetido"}
    c.execute(
```

```
        "INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
(?,?,?,?,?)",
        (
            data["nombre"],
            data["apellido"],
            data["legajo"],
            data.get("email", ""),
            data.get("carrera", CARRERA_POR_DEFECTO),
        ),
    )
    conn.commit()
    return {
        "id": c.lastrowid,
        "nombre": data["nombre"],
        "apellido": data["apellido"],
        "legajo": data["legajo"],
        "email": data.get("email", ""),
        "carrera": data.get("carrera", CARRERA_POR_DEFECTO),
    }


def validar_estudiante(data):
    """Devuelve el mensaje de error del primer dato inválido, o None si están bien."""
    if "nombre" not in data or data["nombre"] == "" or data["nombre"] is None:
        return "falta nombre"
    if "apellido" not in data or data["apellido"] == "":
        return "falta apellido"
    if "legajo" not in data or data["legajo"] == "":
        return "falta legajo"
    if email_invalido(data):
        return "email invalido"
    return None


def legajo_existe(conn, legajo):
    """Indica si ya hay un estudiante con ese legajo."""
    c = conn.cursor()
    c.execute("SELECT * FROM estudiantes WHERE legajo = ?", (legajo,))
    return c.fetchone() is not None


def insertar_estudiante(conn, data):
    """Guarda el estudiante y devuelve el registro creado."""
    estudiante = {
        "nombre": data["nombre"],
        "apellido": data["apellido"],
        "legajo": data["legajo"],
        "email": data.get("email", ""),
        "carrera": data.get("carrera", CARRERA_POR_DEFECTO),
    }
    c = conn.cursor()
    c.execute(
        "INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES
(?,?,?,?,?)",
        (
            estudiante["nombre"],
            estudiante["apellido"],
            estudiante["legajo"],
            estudiante["email"],
            estudiante["carrera"],
        ),
    )
    conn.commit()
```

```
    return {"id": c.lastrowid, **estudiante}
```
ahi?

Claude 06/11/2026 20:20

Todavía no: borraste la versión **nueva** (la corta, al final) y quedó la **vieja**, la que tiene todas las validaciones adentro. Y sigue faltando `listar_estudiantes`.

Para no seguir dando vueltas, reemplazá **todo** el contenido de `app/routers/estudiantes.py` por esto, que ya está en el formato de tu editor:

```
"""Estudiantes: alta y listado."""

import sqlite3

from fastapi import APIRouter, Body, Depends

from app.db import get_db
from app.dominio import CARRERA_POR_DEFECTO
from app.validaciones import email_invalido

router = APIRouter()


def fila_a_estudiante(fila):
    """Convierte una fila de la tabla estudiantes en el diccionario de respuesta."""
    return {
        "id": fila["id"],
        "nombre": fila["nombre"],
        "apellido": fila["apellido"],
        "legajo": fila["legajo"],
        "email": fila["email"],
        "carrera": fila["carrera"],
    }


def validar_estudiante(data):
    """Devuelve el mensaje de error del primer dato inválido, o None si están bien."""
    if "nombre" not in data or data["nombre"] == "" or data["nombre"] is None:
        return "falta nombre"
    if "apellido" not in data or data["apellido"] == "":
        return "falta apellido"
    if "legajo" not in data or data["legajo"] == "":
        return "falta legajo"
    if email_invalido(data):
        return "email invalido"
    return None


def legajo_existe(conn, legajo):
    """Indica si ya hay un estudiante con ese legajo."""
    c = conn.cursor()
    c.execute("SELECT * FROM estudiantes WHERE legajo = ?", (legajo,))
    return c.fetchone() is not None


def insertar_estudiante(conn, data):
    """Guarda el estudiante y devuelve el registro creado."""
    estudiante = {
        "nombre": data["nombre"],
        "apellido": data["apellido"],
        "legajo": data["legajo"],
```

```
        "email": data.get("email", ""),
        "carrera": data.get("carrera", CARRERA_POR_DEFECTO),
    }
    c = conn.cursor()
    c.execute(
        "INSERT INTO estudiantes (nombre, apellido, legajo, email, carrera) VALUES (?,?,?,?,?)",
        (
            estudiante["nombre"],
            estudiante["apellido"],
            estudiante["legajo"],
            estudiante["email"],
            estudiante["carrera"],
        ),
    )
    conn.commit()
    return {"id": c.lastrowid, **estudiante}


@router.post("/estudiantes")
def crear_estudiante(
    data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)
):
    error = validar_estudiante(data)
    if error:
        return {"error": error}
    if legajo_existe(conn, data["legajo"]):
        return {"error": "legajo repetido"}
    return insertar_estudiante(conn, data)


@router.get("/estudiantes")
def listar_estudiantes(conn: sqlite3.Connection = Depends(get_db)):
    c = conn.cursor()
    c.row_factory = sqlite3.Row
    c.execute("SELECT * FROM estudiantes")
    return [fila_a_estudiante(fila) for fila in c.fetchall()]
```

Verificá:

```
Select-String -Path app\routers\estudiantes.py -Pattern "^def |^@router"
.\run.bat test
```

Tiene que listar, en este orden: `fila_a_estudiante`, `validar_estudiante`, `legajo_existe`, `insertar_estudiante`, `@router.post`, `crear_estudiante`, `@router.get` y `listar_estudiantes`. Los tests tienen que dar **64 passed**. Después commiteá el paso 5:

```
git add app/routers/estudiantes.py
git commit -m "refactor: dividir crear_estudiante en validacion, busqueda de legajo e insercion"
```

Usuario (Cristian Morales) 06/11/2026 20:20

app\routers\estudiantes.py:14:def fila_a_estudiante(fila):
app\routers\estudiantes.py:26:def validar_estudiante(data):
app\routers\estudiantes.py:39:def legajo_existe(conn, legajo):
app\routers\estudiantes.py:46:def insertar_estudiante(conn, data):
app\routers\estudiantes.py:70:@router.post("/estudiantes")
app\routers\estudiantes.py:71:def crear_estudiante(
app\routers\estudiantes.py:82:@router.get("/estudiantes")
app\routers\estudiantes.py:83:def listar_estudiantes(conn: sqlite3.Connection = Depends(get_db)):

Claude 06/11/2026 20:20

Perfecto: cada función aparece una sola vez y en el orden correcto. El paso 5 quedó bien.

Ahora:

```
.\run.bat test
git add app/routers/estudiantes.py
git commit -m "refactor: dividir crear_estudiante en validacion, busqueda de legajo e insercion"
```

Los tests tienen que dar **64 passed**. Después seguí con el **paso 6**, la documentación (los dos `.md` que te mandé), y cerrá con la verificación final: 6 commits, árbol limpio, 64 passed y push.

Usuario (Cristian Morales) 06/11/2026 20:20

dame el paso 6

Claude 06/11/2026 20:20

▸ *Envió los archivos cristian_refactoring.md, cristian.md*

Te mandé los dos archivos de nuevo:

- `cristian_refactoring.md` documenta el antes y después de cada refactoring (los transversales y los 5 de tu parte) y explica por qué los hallazgos 3, 5 y 11 quedan como corrección de defecto. Va en **`docs/refactoring/`**.
- `cristian.md` es tu declaración de uso de IA con la fila de la Fase 3 agregada. **Reemplaza** a `docs/uso-ia/cristian.md`.

Paso 6

1. Fijate cómo se guardaron en Descargas, por si Windows les agregó `(1)`:

```
Get-ChildItem "$env:USERPROFILE\Downloads" -Filter "cristian*.md"
```

2. Copialos al repo. Si los nombres tienen `(1)`, ajustá el origen; el destino queda igual:

```
New-Item -ItemType Directory -Force docs\refactoring | Out-Null
Copy-Item "$env:USERPROFILE\Downloads\cristian_refactoring.md"
docs\refactoring\cristian_refactoring.md -Force
Copy-Item "$env:USERPROFILE\Downloads\cristian.md" docs\uso-ia\cristian.md -Force
```

3. Verificá antes de commitear:

```
git status
Select-String -Path docs\uso-ia\cristian.md -Pattern "Refactoring \(Fase 3\)"
```

- El `status` tiene que mostrar `docs/refactoring/` como nuevo y `docs/uso-ia/cristian.md` como modificado, y nada más.
- El `Select-String` tiene que encontrar la fila nueva.

4. Commiteá:

```
git add docs/refactoring/ docs/uso-ia/cristian.md
git commit -m "docs: refactorings de la fase 3 (antes y despues) y registro de uso de IA"
```

Cierre

```
git log --oneline dev..HEAD
git status
.\run.bat test
git push -u origin refactor/personas
```

Tienen que aparecer **6 commits**, el árbol tiene que estar limpio y los tests tienen que dar **64 passed**. Después abrí el PR a `dev` y pasame el link.

Usuario (Cristian Morales) 06/11/2026 20:22

git log --oneline dev..HEAD
00838d8 (HEAD -> refactor/personas) docs: refactorings de la fase 3 (antes y despues) y registro de uso de IA
8cfc259 refactor: dividir crear_estudiante en validacion, busqueda de legajo e insercion
6d4427d refactor: centralizar las areas y la carrera por defecto en app/dominio.py
be3923d refactor: extraer la validacion de email duplicada a app/validaciones.py
ed6f891 refactor: extraer la conversion de filas y acceder a las columnas por nombre
6d9cf25 refactor: renombrar los endpoints de responsables con el criterio de estudiantes
git status
On branch refactor/personas
nothing to commit, working tree clean
.\run.bat test
========== test session starts ==========
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 64 items

tests\test_asignacion.py .... [6%]
tests\test_buscar.py ..... [25%]
tests\test_detalle.py ..... [34%]
tests\test_estado.py ..... [42%]
tests\test_estudiantes.py ..... [57%]
tests\test_reporte.py ..... [68%]
tests\test_responsables.py ... [73%]
tests\test_solicitudes.py ..... [100%]

========== warnings summary ==========
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========== 64 passed, 1 warning in 1.37s ==========
git push -u origin refactor/personas
Enumerating objects: 45, done.
Counting objects: 100% (45/45), done.
Delta compression using up to 16 threads
Compressing objects: 100% (37/37), done.
Writing objects: 100% (37/37), 7.47 KiB | 2.49 MiB/s, done.
Total 37 (delta 25), reused 0 (delta 0), pack-reused 0 (from 0)

remote: Resolving deltas: 100% (25/25), completed with 5 local objects.
remote:
remote: Create a pull request for 'refactor/personas' on GitHub by visiting:
remote:
https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta/pull/new/refactor/personas
remote:
To https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta.git
 * [new branch]   refactor/personas -> refactor/personas

Claude 06/11/2026 20:23

▸ *Ejecutó un comando: Correr los tests en cada commit subido*

▸ *Ejecutó un comando: Comparar los documentos subidos*

Revisé la rama en GitHub, commit por commit, y quedó bien.

Commit	Tests	Cambios
`6d9cf25` renombrar endpoints	64 passed	1 archivo
`ed6f891` conversión de filas	64 passed	2 archivos
`be3923d` validación de email	64 passed	3 archivos
`6d4427d` constantes de dominio	64 passed	3 archivos
`8cfc259` dividir `crear_estudiante`	64 passed	1 archivo
`00838d8` documentación	64 passed	2 archivos

- **Cada commit deja los tests en verde por sí solo.** Es lo que demuestra "refactoring seguro" en el historial, no solo al final.
- **El código está bien:** `responsables.py` usa `AREAS` y `email_invalido`, `estudiantes.py` quedó con las funciones en orden, y no hay imports sin usar.
- **Los documentos** coinciden con los que te mandé. La única diferencia es el salto de línea final, que no importa.
- **No se coló nada:** sin `(1)` y sin `__pycache__`.

Ahora

1. Abrí el PR desde el link que te dio el push, con base **`dev`**.
2. **Título:** `Fase 3: refactoring de estudiantes y responsables`
3. **Descripción:**

``markdown Refactorings de estudiantes y responsables (hallazgos 4, 6, 7, 8, 9, 12 y 13 de Cristian). El detalle con antes y después está en`docs/refactoring/cristian_refactoring.md`.

- 5 commits de refactoring, uno por técnica; cada uno deja los 64 tests en verde.
- Ningún assert modificado.
- Los hallazgos 3, 5 y 11 cambian el comportamiento: quedan como corrección de defecto, para cuando el grupo unifique el criterio de códigos HTTP.
- `app/dominio.py` (`AREAS`) y `app/validaciones.py` quedan disponibles para el resto: `registrar` puede reutilizar `AREAS`. ```

4. Pedile la revisión a alguien del grupo y mergealo con **"Create a merge commit"**.

Avisale a Gaston que `AREAS` ya está en `app/dominio.py`, así lo usa en `registrar` en vez de repetir la lista. Es justo el hallazgo 13 (Shotgun Surgery).

Usuario (Cristian Morales) 06/11/2026 21:54

revisa el repo y avisame como venimos y que falta

Claude 06/11/2026 21:54

▸ *Ejecutó un comando: Revisar el estado del repositorio*

▸ *Ejecutó un comando: Correr los tests en dev y revisar el refactor de Bruno*

▸ *Ejecutó un comando: Revisar las declaraciones de uso de IA*

Revisé `dev`. Vamos bien: los cuatro tienen el diagnóstico y los tests, la estructura nueva está integrada, y dos de los cuatro ya terminaron su refactor. En `dev` corren **67 tests, todos en verde**.

Cómo venimos

Etapa	Estado
Diagnóstico	✓ cerrada, tag `etapa-diagnostico` en `main`
Tests de línea de base	✓ cerrada, tag `etapa-tests` en `main` (64 tests)
Refactoring	**en curso**, todavía en `dev`

El refactoring, por integrante:

Integrante	Router	Estado
Cristian	transversales + `estudiantes` / `responsables`	✓ mergeado (#13 y #18), con su documento de antes y después
Bruno	`consultas.py`	✓ mergeado (#17): Extract Method. Además sumó 3 tests nuevos para el cálculo de vencidas, que no tenía cobertura. Por eso son 67
Gaston	`solicitudes.py`	✗ **sin empezar**: el archivo sigue igual que cuando lo moviste
Javier	`estados.py`	✗ **sin empezar**

Los dos que faltan son los que garantizan dos de los tres refactorings obligatorios: **Introduce Parameter Object** (Gaston) y **Move Method** (Javier). Sin ellos no se cumple la consigna, así que es lo más urgente.

Lo que falta

Para cerrar la etapa de refactoring:

1. **Gaston y Javier:** su refactor, con PR a `dev`. Gaston puede usar `AREAS` de `app/dominio.py` en `registrar`.
2. **Bruno** no tiene un documento de refactoring en `docs/refactoring/` (solo la captura de la suite). Que agregue uno con antes y después, como el tuyo, porque la consigna lo pide.
3. **Las correcciones de defecto** (de 200 a 422, nulos, inyección SQL...). Primero el grupo tiene que unificar el criterio de códigos HTTP: 404 o 422 para un recurso del body que no existe, y cómo evaluar el body como lista. Después van en commits `fix:` aparte, actualizando los asserts afectados.
4. Recién ahí: PR `dev` → `main` y tag **`etapa-refactoring`**.

Pendientes de las etapas anteriores (ninguno se resolvió todavía):

- **El PDF de IA de Bruno:** su `bruno.md` cita `anexo/ia/bruno_claude_01_diagnostico.pdf`, que no está.
- **`docs/uso-ia/gaston.md`:** la segunda fila de la tabla sigue con 3 columnas en vez de 5.
- **Los PRs #5 y #6** siguen abiertos desde octubre. Si no hacen falta, que los cierren.

Tuyo:

- `anexo/ia/cristian_claude_04_refactoring.pdf`: tu fila de la Fase 3 lo cita. Exportá la conversación cuando cierres la etapa y te armo el PDF.

Lo que viene después (etapas siguientes):

- **`etapa-pruebas`:** la verificación final. Correr la suite completa sobre el código refactorizado y documentarla con evidencias.
- **`etapa-docs`:** el informe en PDF y el checklist de code review.
- **`estadorefactorizado`**, sobre `main`, con la versión final.

Te sugiero mandar hoy al grupo un mensaje con dos cosas: que Gaston y Javier arranquen sus refactors, y la propuesta para unificar el criterio de códigos HTTP. ¿Querés que te lo redacte?

Usuario (Cristian Morales) 07/11/2026 21:39

bien, revisa el repo que falta

Claude 07/11/2026 21:40

▸ *Ejecutó un comando: Revisar el estado del repositorio*

▸ *Ejecutó un comando: Revisar el refactor de Gaston y los pendientes*

▸ *Ejecutó un comando: Revisar app/dominio.py y la declaración de Gaston*

Revisé `dev`: desde ayer se mergeó el refactor de Gaston, y ya son **74 tests en verde**. Solo falta Javier para cerrar los refactorings.

Cómo venimos

Integrante	Refactor	Estado
Cristian	transversales + estudiantes/responsables	✓ #13 y #18, con documento
Bruno	`consultas.py` (Extract Method)	✓ #17, **sin documento**
Gaston	`solicitudes.py`	✓ #19, con documento (349 líneas)
Javier	`estados.py`	✗ **sin empezar**

El de Gaston está muy completo: **Introduce Parameter Object** con un dataclass `NuevaSolicitud`, `REGLAS_POR_TIPO` en `app/dominio.py` en lugar de los `if` por tipo, la notificación y el historial extraídos, y 7 tests nuevos de prioridad y área. No modificó ningún assert existente.

Obligatorios de la consigna:

- Extract Method ✓ (Bruno)
- Introduce Parameter Object ✓ (Gaston)
- **Move Method ✗: depende de Javier.** Es lo único que falta para cumplir los tres.

Qué falta

Bloquea el cierre de la etapa de refactoring:

1. **Javier: su refactor de `estados.py`, con el Move Method.** Candidatos claros: mover las transiciones de estado a `app/dominio.py` (como hizo Gaston con las reglas por tipo), o mover la notificación a un módulo compartido, porque Gaston ya extrajo `notificar_alta` y la de Javier hace lo mismo.
2. **Bruno: `docs/refactoring/bruno_refactoring.md`** con el antes y después. Él y Javier son los únicos sin documento.

Pendientes de las etapas anteriores, todavía sin resolver:

3. **El PDF de IA de Bruno** (`anexo/ia/bruno_claude_01_diagnostico.pdf`): sigue sin estar. Va a necesitar también el de la Fase 3.
4. **`docs/uso-ia/gaston.md`:** la fila del diagnóstico sigue con 3 columnas. Gaston tocó el archivo en este PR y no lo corrigió. Le falta la columna de evidencia al final (`| anexo/ia/gaston_registro_ia.md |`).
5. **Los PRs #5 y #6**, abiertos desde octubre.

Decisión del grupo antes de cerrar:

6. **Las correcciones de defecto.** ¿Se hacen en esta etapa (commits `fix:` que cambian 200 por 422 y actualizan los asserts) o quedan documentadas como deuda técnica? Cualquiera de las dos es válida, pero tiene que quedar escrita en el informe.

Tuyo:

7. **`anexo/ia/cristian_claude_04_refactoring.pdf`**: cuando cierre la etapa, exportás la conversación y te lo armo.

Limpieza (opcional, pero suma prolijidad): hay 17 ramas viejas ya mergeadas (`docs/diagnostico-*`, `test/*`, `refactor/*`, los `*-patch-1`...). Se pueden borrar desde GitHub, en **Branches**. Los commits quedan en `dev`.

Para cerrar la etapa

Cuando estén el refactor de Javier y el documento de Bruno, y el grupo haya decidido el punto 6:

1. PR `dev` → `main`, "*Fase 3: refactoring*", con merge commit.
2. Tag **`etapa-refactoring`** sobre `main`.

Lo urgente es Javier. ¿Querés que te arme un mensaje para el grupo con los puntos 1 a 6?

Usuario (Cristian Morales) 07/11/2026 23:09

como quedo el repo?

Claude 07/11/2026 23:10

▸ *Ejecutó un comando: Revisar el estado del repositorio*

▸ *Ejecutó un comando: Revisar los PRs #21 y #22*

▸ *Ejecutó un comando: Verificar las evidencias de IA y la compatibilidad de los PRs*

Muy bien: **ya están los cuatro refactors.** Los últimos dos están en PRs abiertos, listos para mergear.

Los PRs nuevos

PR	Autor	Contenido	Revisión
#22	Javier	Refactor de `estados.py`: 5 commits + documento	✓ sin conflictos, 74 passed, ningún assert tocado
#21	Bruno	Su documento de refactoring, los resultados de la Fase 2 y el registro de uso de IA	✓ sin conflictos, 74 passed

Los dos se pueden mergear en cualquier orden: no se pisan entre sí.

El de Javier incluye el Move Method que faltaba: mueve las transiciones de estado y los estados cerrados a `app/dominio.py` (`TRANSICIONES`, `ESTADOS_CERRADOS`, `transicion_permitida()`). Además, lee las columnas por nombre, extrae el historial y la notificación, y renombra `proc_estado` → `cambiar_estado` y `asignar` → `asignar_responsable`.

El de Bruno resuelve su pendiente de IA: en lugar del PDF que faltaba, ahora cita `anexo/ia/bruno_registro_ia.md`, y ese archivo está en el PR.

Cómo queda el refactoring

Integrante	Refactor	Documento
Cristian	✓ #13 y #18	✓
Bruno	✓ #17	✓ en #21
Gaston	✓ #19	✓
Javier	✓ en #22	✓ en #22

Con #21 y #22 mergeados, **se cumplen los tres obligatorios**: Extract Method (Bruno), Introduce Parameter Object (Gaston) y Move Method (Javier).

Lo que falta

Para cerrar `etapa-refactoring`:

1. **Revisar y mergear #21 y #22**, con merge commit.
2. **La decisión de las correcciones de defecto:** ¿se hacen ahora (commits `fix:` que cambian los asserts) o quedan documentadas como deuda técnica? Tiene que quedar escrito.
3. **Tu `anexo/ia/cristian_claude_04_refactoring.pdf`:** exportá la conversación y te lo armo.
4. Después, PR `dev` → `main` y tag `etapa-refactoring`.

Pendientes chicos:

- **`docs/uso-ia/gaston.md`:** la fila del diagnóstico sigue con 3 columnas.
- **Los PRs #5 y #6**, abiertos desde octubre: cerrarlos si no hacen falta.

Para las etapas siguientes:

- **El checklist de code review no existe todavía en `docs/`.** La consigna lo pide. Lo coordina Javier.
- La verificación final (`etapa-pruebas`), el informe en PDF (`etapa-docs`), el ZIP de anexos y el tag final `estadorefactorizado`.